Securing IoT Devices in Smart Home Environments

Project Report

Prepared for First Quadrant Labs

Thamizh Selvan M.R.

24 October 2026

Summary

I built a four-VM smart home lab, attacked it in its default state, then hardened it layer by layer and tested each control. The baseline broker accepted anonymous connections, sent everything in plaintext, and let me spoof an "UNLOCKED" command to a door lock. After hardening, none of that worked.

- **Network:** three separate networks behind an nftables firewall with a default-drop policy. Port 1883 is closed on the broker.
- **Authentication:** MQTT over TLS on port 8883 with mutual certificate auth, using my own CA. Connections with no certificate or the wrong CA are rejected.
- **Authorization:** an ACL limits each device to its own topics. A denied publish is proven by "messages delivered: 0".
- **Data at rest:** a LUKS2 vault on the broker and an encrypted cache on the device.
- **Firmware:** signed updates. Tampered firmware, a bad signature and a rollback to an older version are all rejected.
- **Monitoring:** Suricata with three custom rules, plus firewall drop logs.
- **Incident response:** I contained a compromised device, revoked its certificate, issued a new one and recovered service.

Secure boot and tamper detection are software simulations. I label them that way wherever they appear.

Introduction and scope

Smart home devices are usually shipped open. Default MQTT brokers allow anonymous clients, traffic is plaintext, and one compromised sensor can reach everything else on the same network. This project asks a simple question: what does it take to make a small smart home setup defensible, and can I prove each control works?

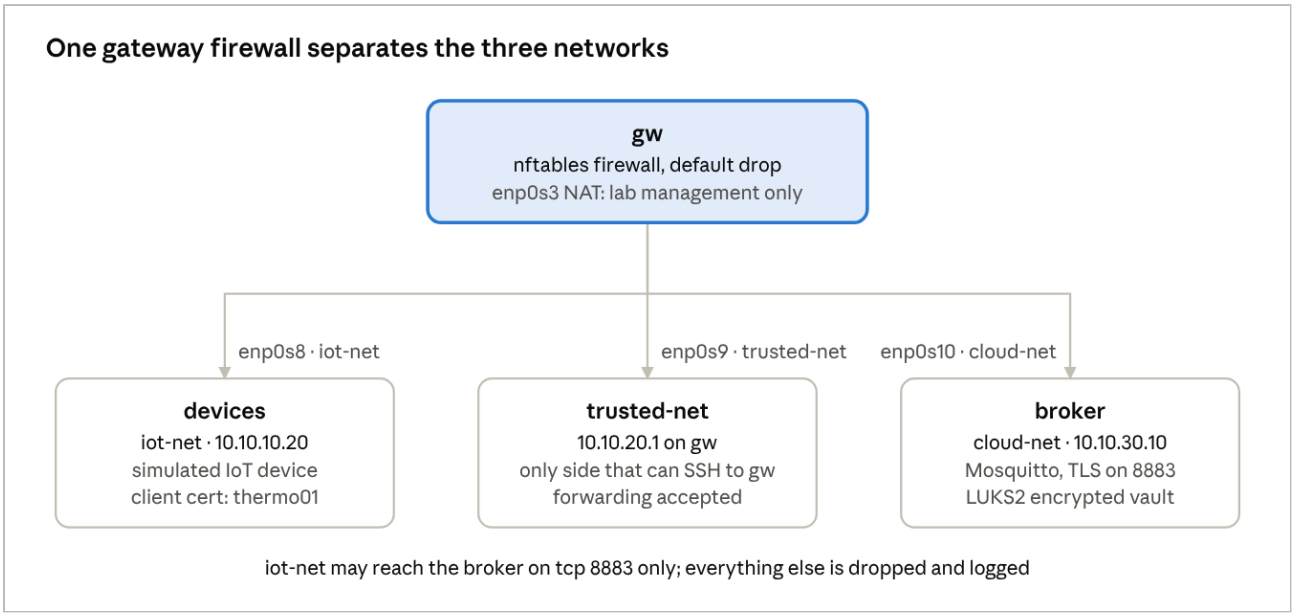
I worked in a closed VirtualBox lab. No real devices, no real home network, and every attack was run only against my own VMs.

The brief covers nine areas: architecture, authentication and authorization, encryption, network security, firmware, physical security, monitoring and incident response, user training, and policy and compliance. Each one has a section below with the test I ran and the screenshot that shows the result.

Method for every control: attack the baseline first, apply the fix, then repeat the same attack and compare.

Lab architecture

Four VirtualBox VMs on internal networks, with one gateway that routes and filters between them. Ubuntu 26.04.1 LTS runs on gw, broker and devices; Kali is the attacker.



lab architecture · 3 networks, 1 gateway

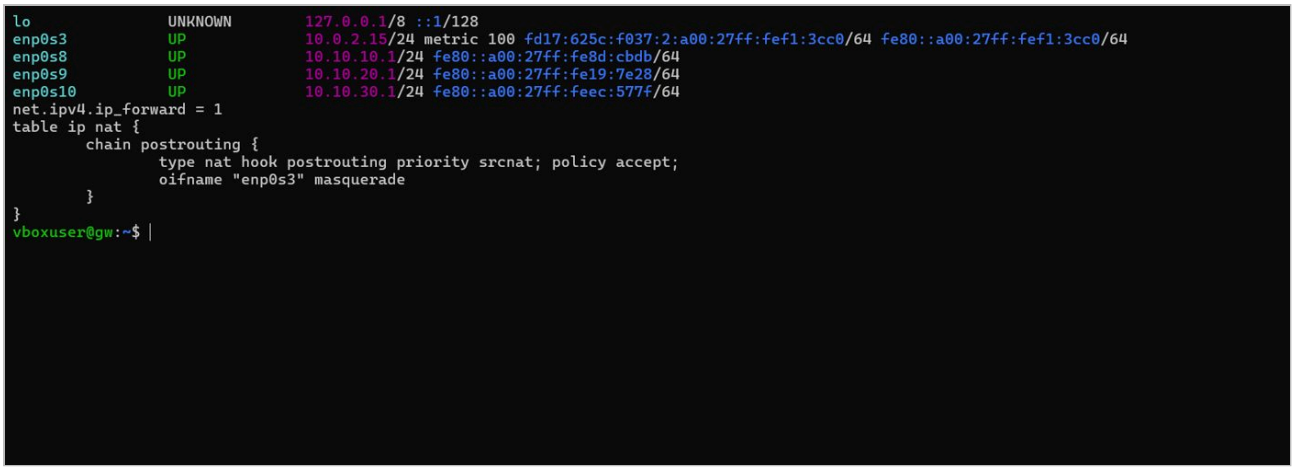
The gateway is the only path between segments, so every cross-network packet passes through the nftables ruleset.

VM	Role	Network	IP
gw	Router, firewall, Suricata sensor	iot-net, trusted-net, cloud-net, NAT	10.10.10.1, 10.10.20.1, 10.10.30.1
broker	MQTT broker (Mosquitto 2.0.22)	cloud-net	10.10.30.10
devices	Simulated IoT device	iot-net	10.10.10.20
kali	Attacker	iot-net	10.10.10.2

broker and devices also have a NAT adapter (Adapter 1). I used it only for SSH through port forwarding (gw 2222, broker 2224, devices 2225) to manage the lab. It is not part of the design.

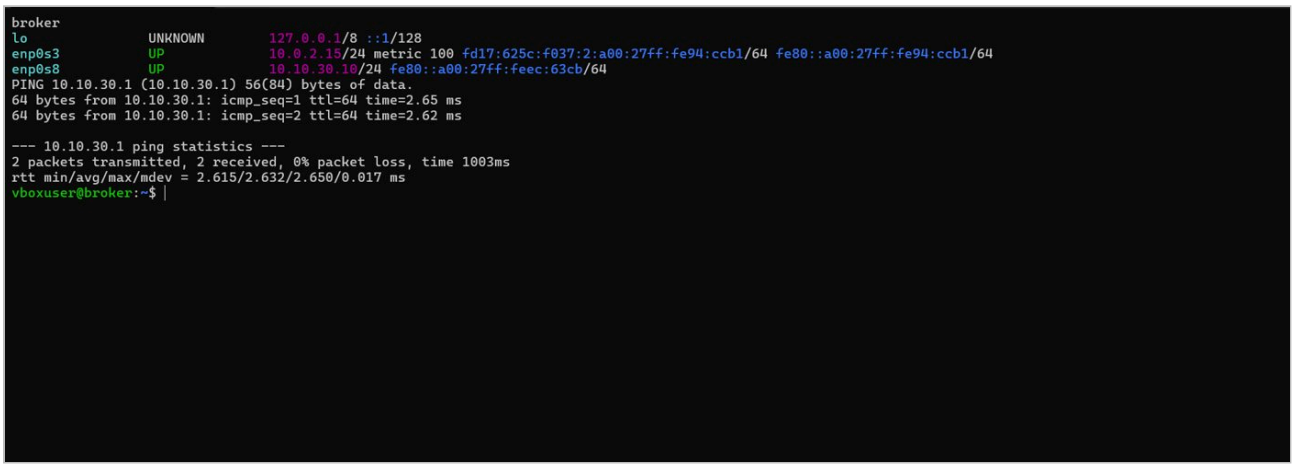
Setup checks

The gateway has all three interface addresses, IP forwarding is on and NAT is set.



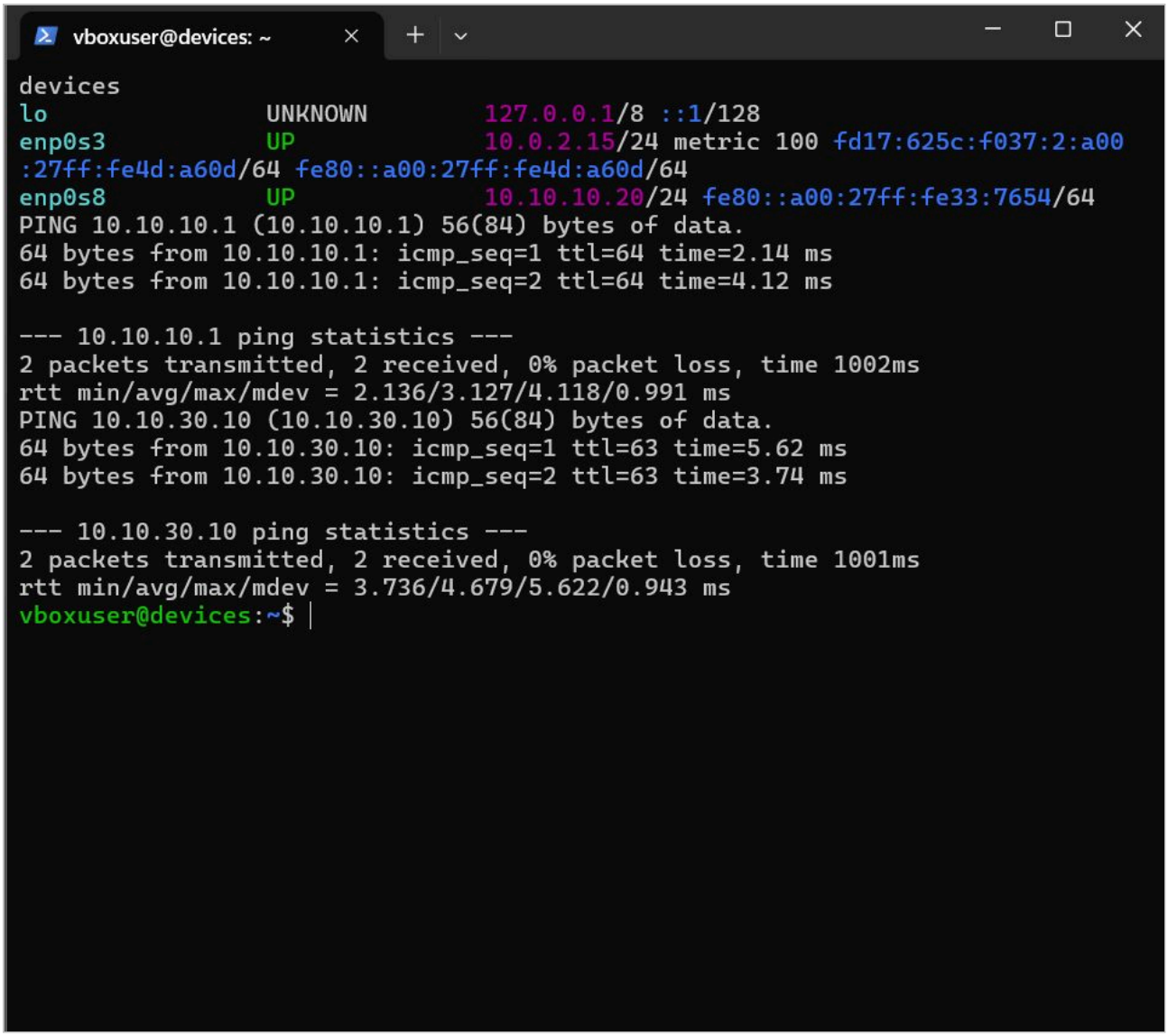
gw interfaces, ip_forward and NAT

From broker, a ping to the gateway on cloud-net succeeds.



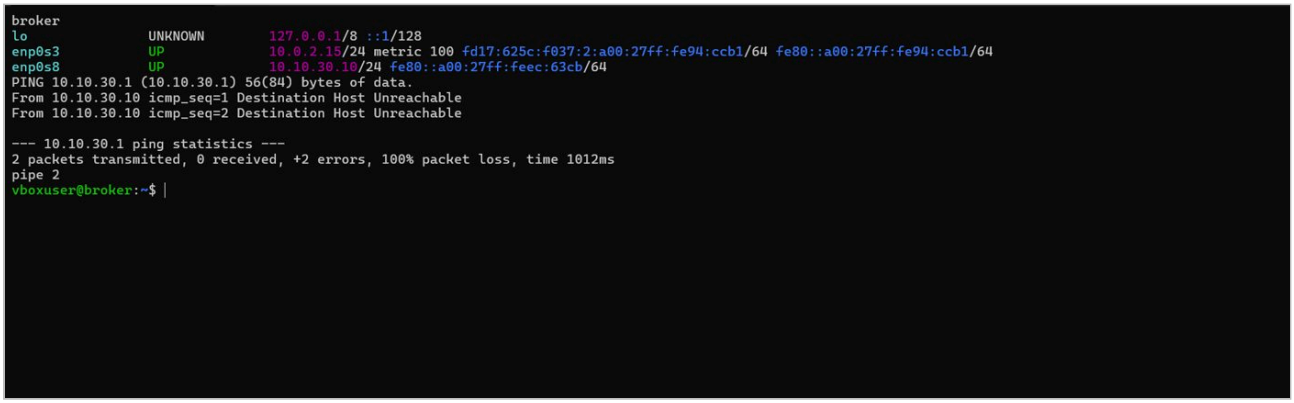
broker pings gw

From devices, pings cross the gateway. The ttl=63 shows the packet went through one router.

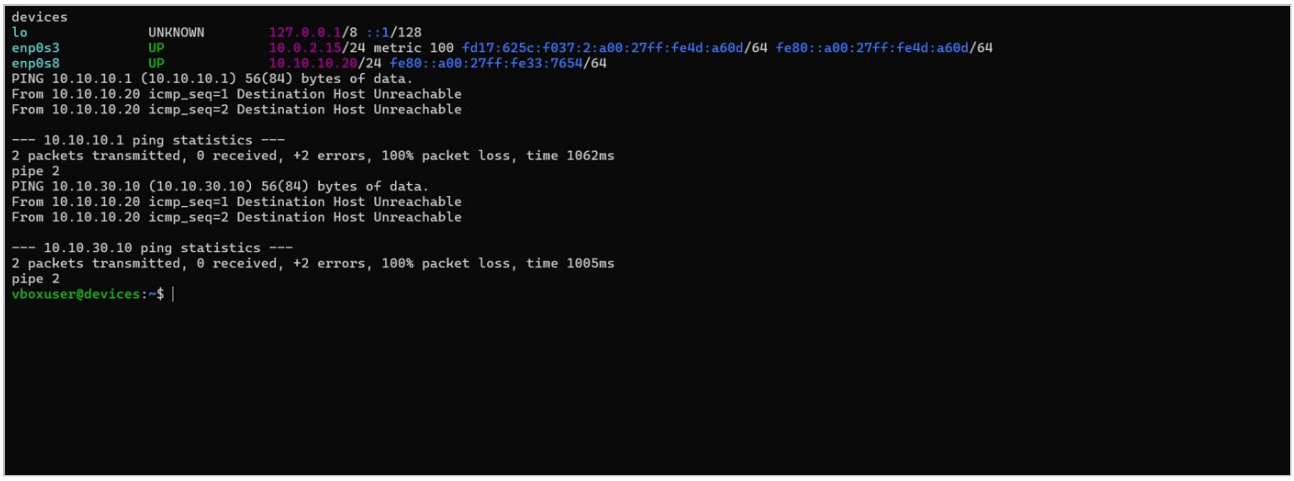


devices ping across the gateway

Two failures I hit. Both pings failed with "Destination Host Unreachable" because gw was powered off. Starting gw fixed both.



broker ping failing while gw was off

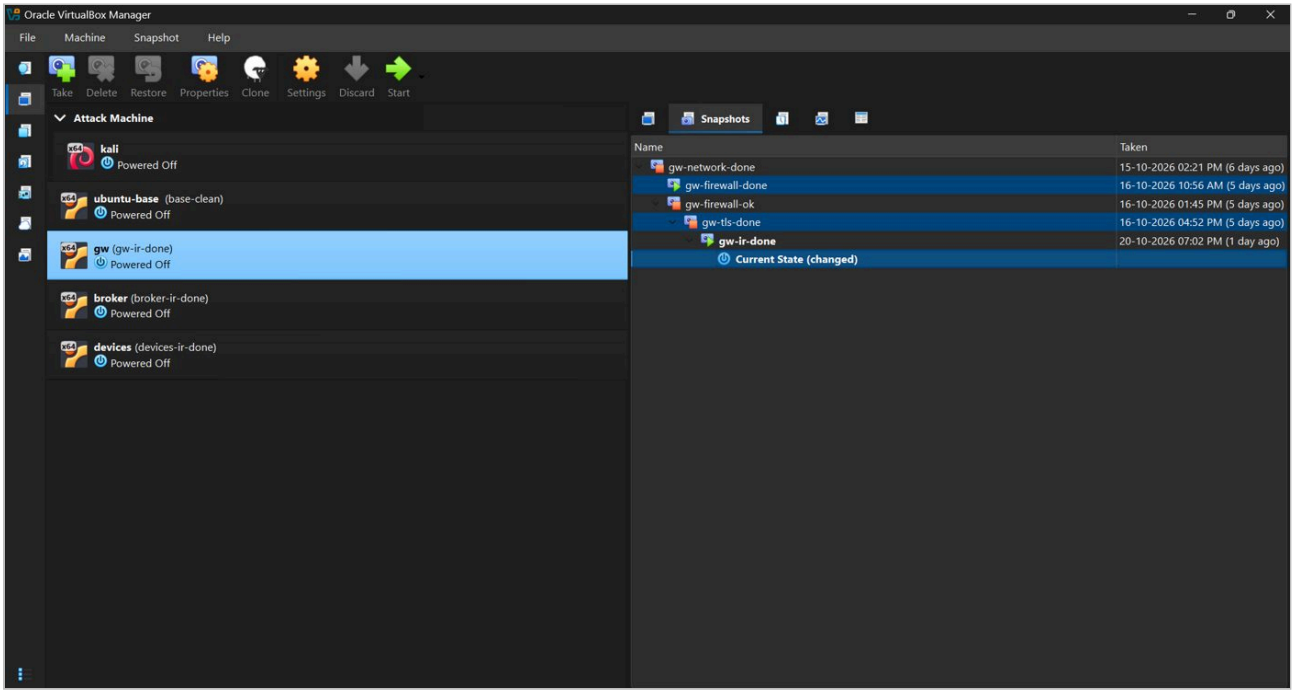


devices ping failing while gw was off

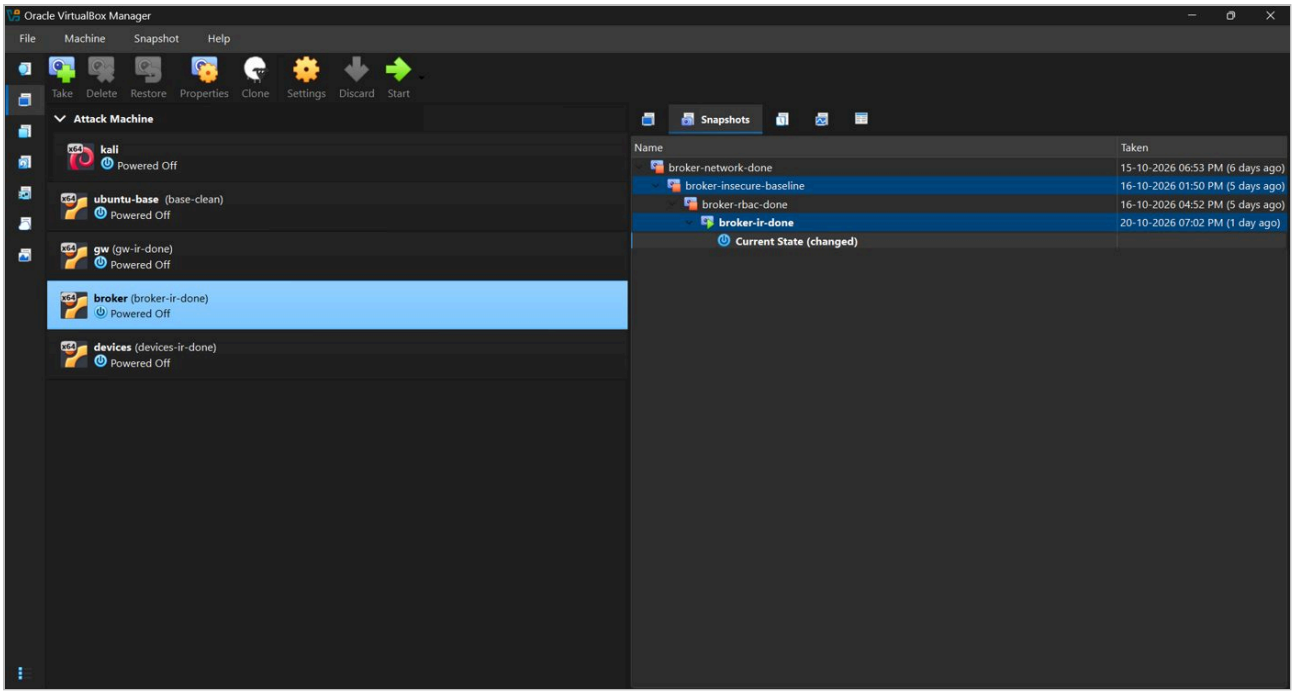
Snapshots

I took a snapshot after each stage on every VM, so any step can be rolled back and re-tested. All four VMs are powered off in these screenshots.

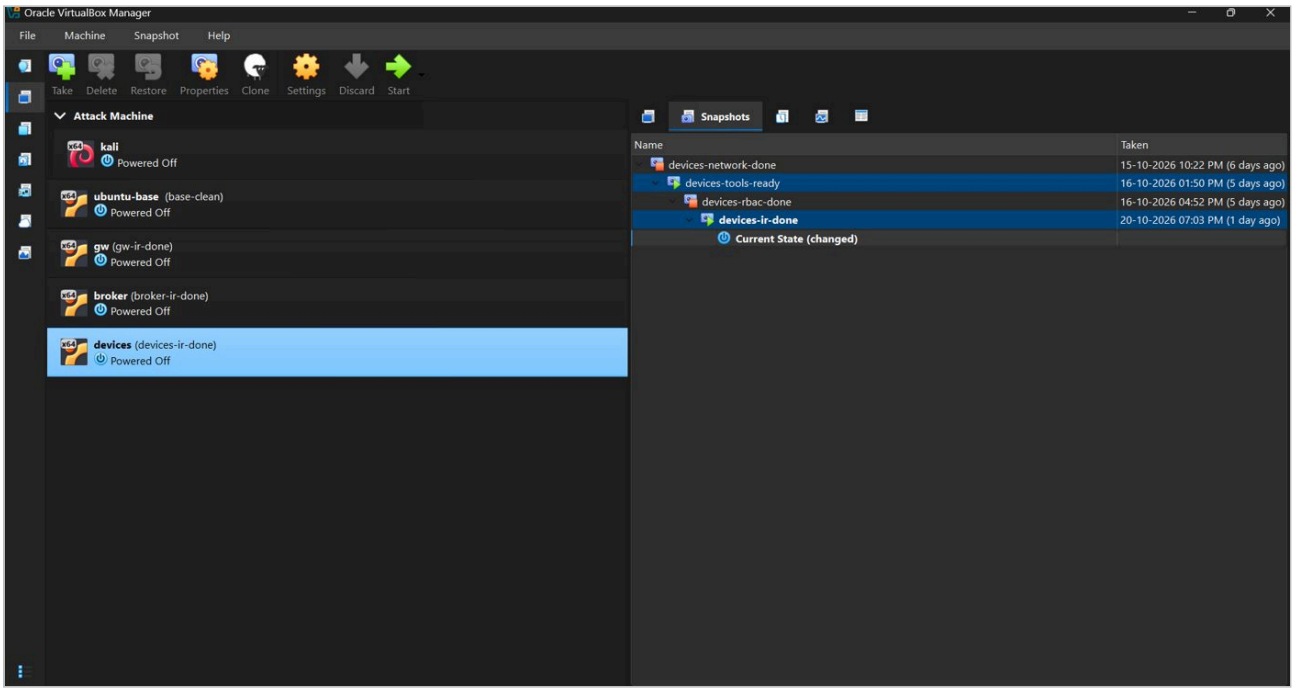
- **gw:** gw-network-done, gw-firewall-done, gw-firewall-ok, gw-tls-done, gw-ir-done
- **broker:** broker-network-done, broker-insecure-baseline, broker-rbac-done, broker-ir-done
- **devices:** devices-network-done, devices-tools-ready, devices-rbac-done, devices-ir-done
- **ubuntu-base:** base-clean, the clean image the three Ubuntu VMs were cloned from



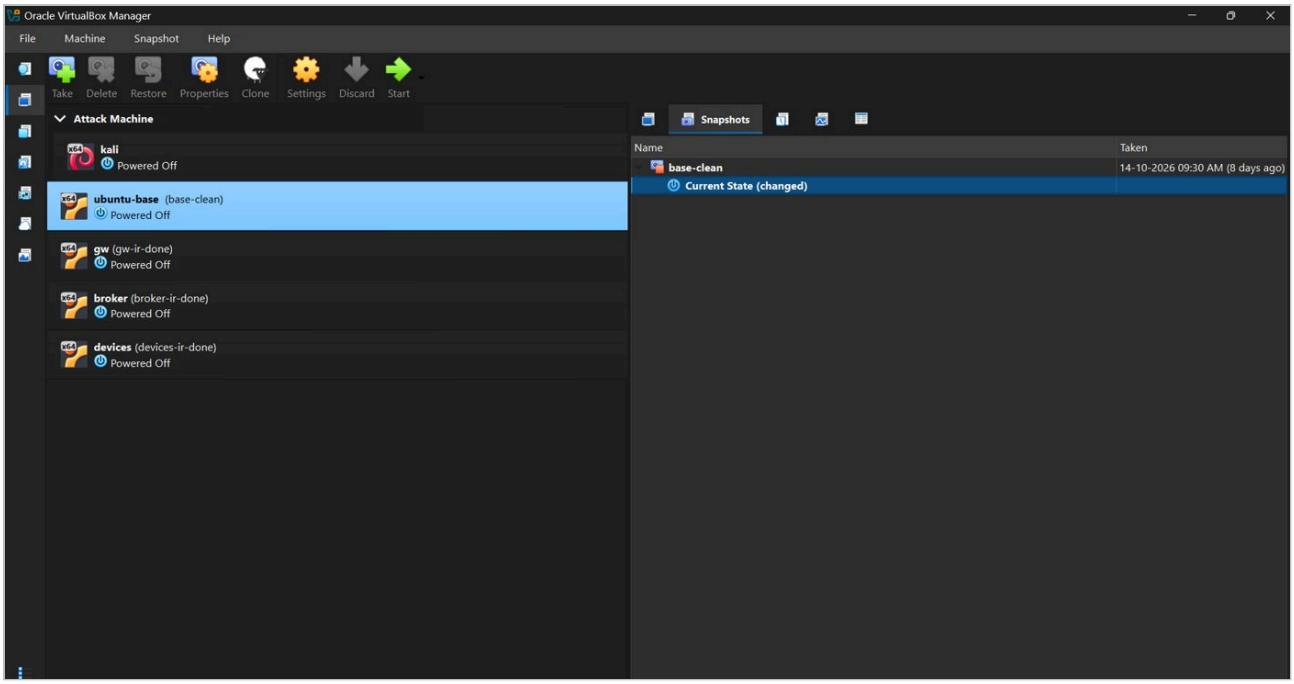
gw snapshots



broker snapshots



devices snapshots

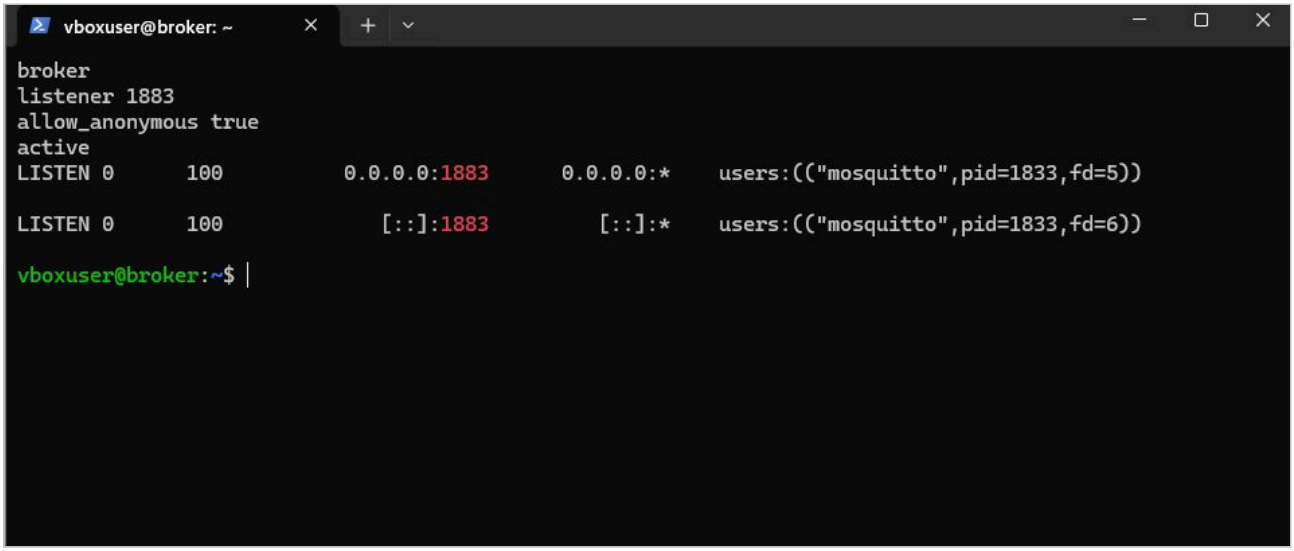


ubuntu-base snapshot

Baseline assessment

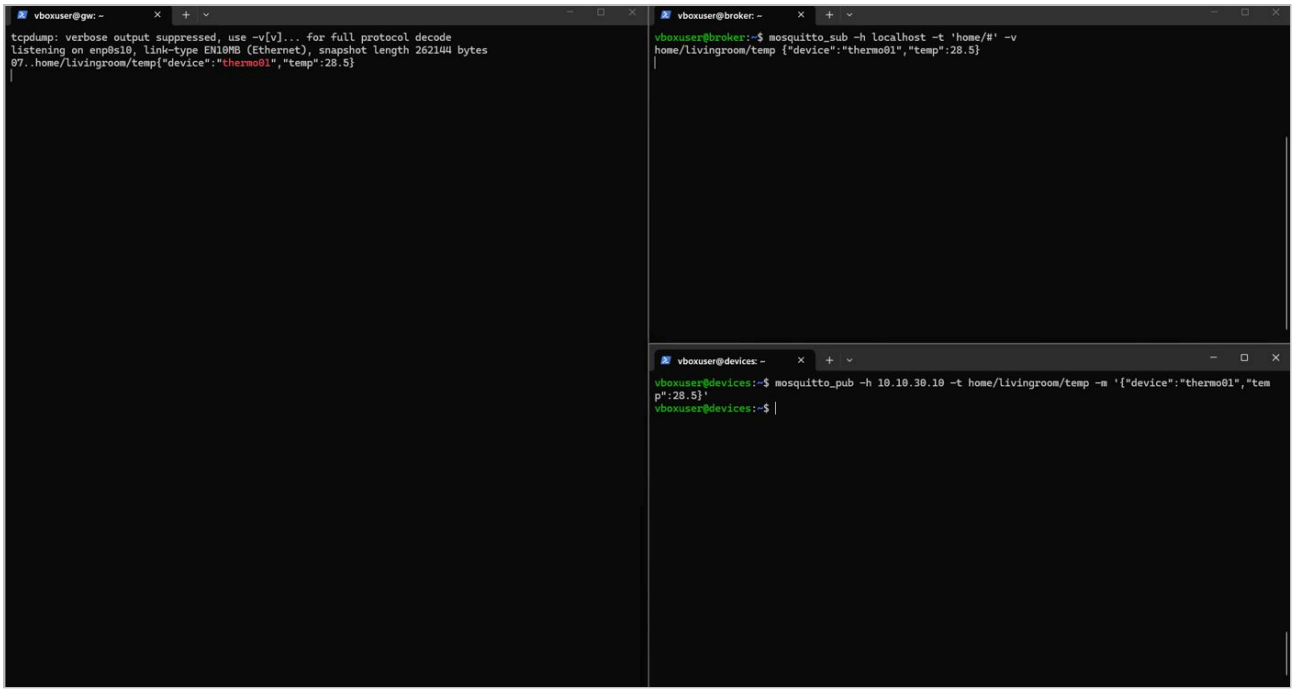
Before any hardening, an attacker on the IoT segment could read, impersonate and open the lock. Four findings, all from the default setup.

1. Anonymous access. The broker listened on 1883 with allow_anonymous true. Any client can connect with no credentials.



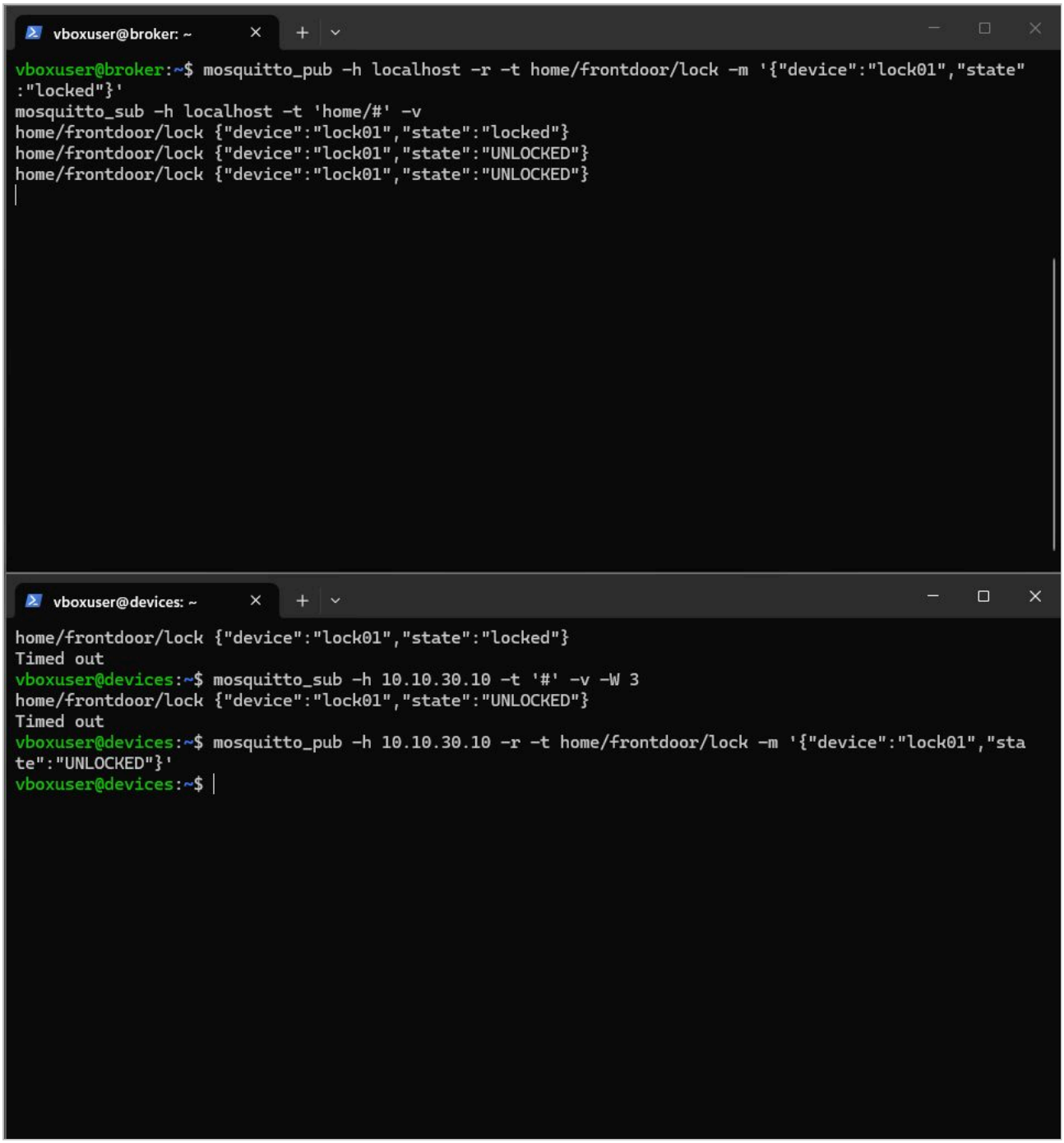
Mosquitto baseline config with allow_anonymous true on 1883

2. Plaintext traffic. A tcpdump on the wire shows topics and payloads in clear text.



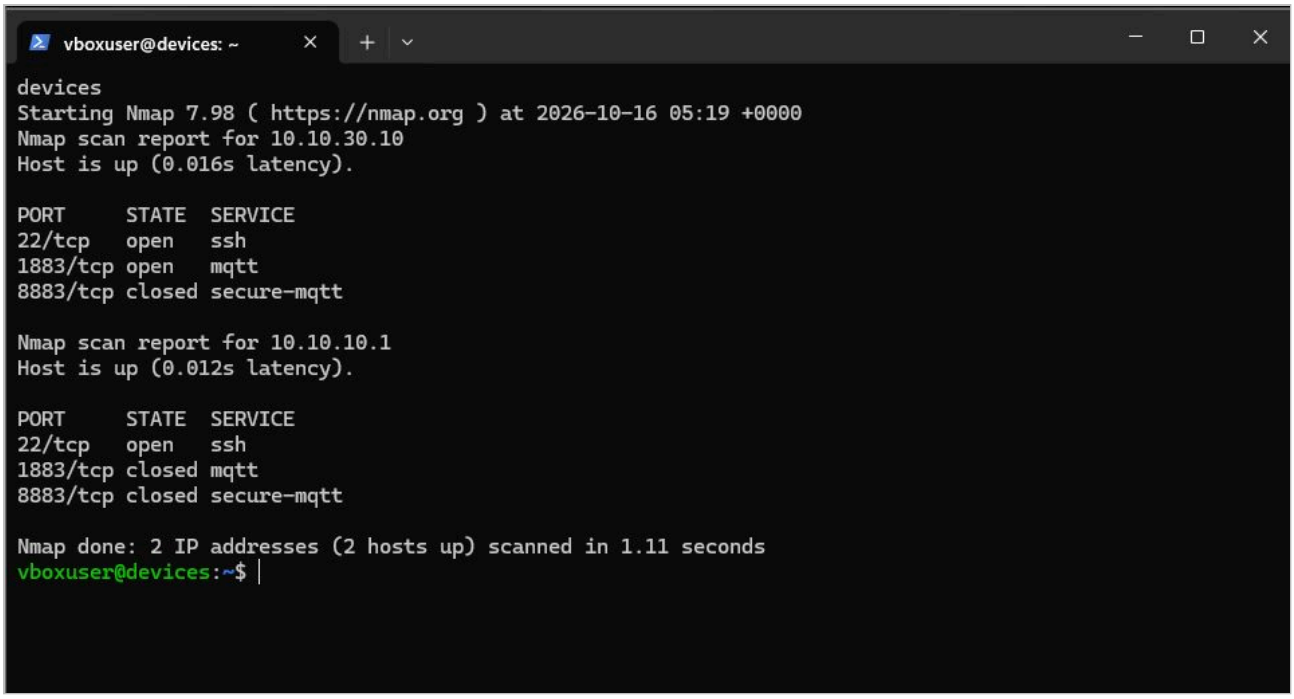
plaintext MQTT visible in tcpdump

3. Command spoofing. From the attacker VM I published an "UNLOCKED" message to the door lock topic with no login. The broker accepted it.



anonymous spoofed UNLOCKED lock message

4. Exposed services. An nmap scan found SSH (22) and MQTT (1883) both open.



nmap showing ports 22 and 1883 open

This is what every later section tries to close.

Findings and recommendations

Finding	Risk (my rating)	Recommendation	Fixed in
Anonymous broker access	High	Require client certificates	TLS and mutual authentication
Plaintext MQTT	High	TLS on 8883, close 1883	TLS and mutual authentication
No topic limits, so any client can spoof the lock	High	Per-identity ACL	Authorization (ACL)
SSH and MQTT open to the IoT segment	Medium	Default-drop firewall, SSH from trusted-net only	Network segmentation and firewall
No detection of scans or plaintext traffic	Medium	Suricata rules, firewall logs	Monitoring and IDS

Network segmentation and firewall

I put the devices, the broker and the trusted side on three separate networks and made the gateway drop everything it was not told to allow. The ruleset is an nftables table called iotfw.

Chain	Policy	What it accepts
input	drop	established and related, loopback, the NAT interface, rate-limited ICMP, SSH only from trusted-net

Chain	Policy	What it accepts
forward	drop	iot-net to broker on tcp 8883, all trusted-net traffic

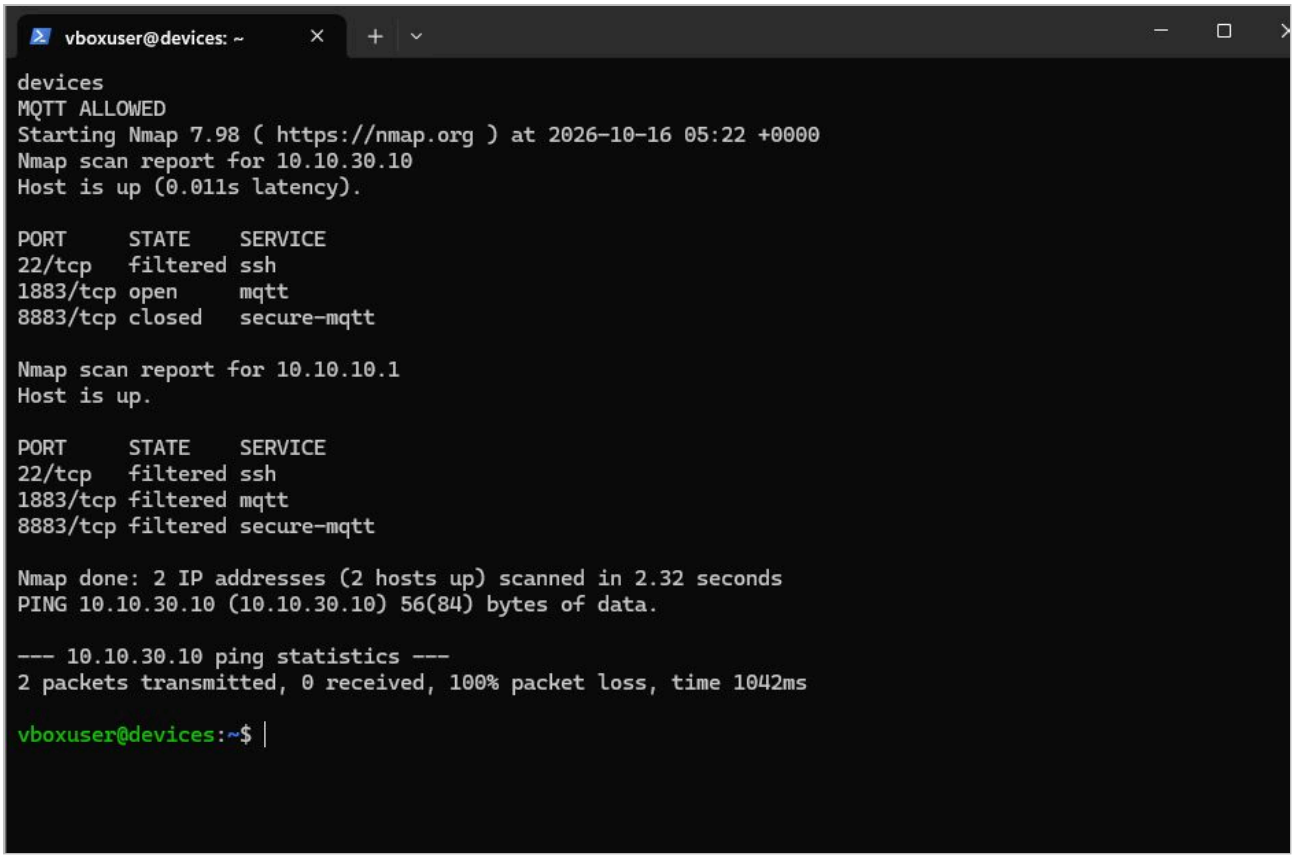
The forward rule first allowed tcp 1883 and 8883. A later scan from Kali showed 1883 was only being stopped by the broker, so I narrowed the rule to 8883 (see the attack tests). Dropped packets are logged with the prefixes FW-INPUT-DROP and FW-FWD-DROP.

```
gw
table inet iotfw {
        chain input {
                type filter hook input priority filter; policy drop;
                ct state established,related accept
                ct state invalid drop
                iif "lo" accept
                iifname "enp0s3" accept
                ip6 nexthdr ipv6-icmp accept
                ip protocol icmp icmp type echo-request limit rate 10/second burst 5 packets accept
                iifname "enp0s9" tcp dport 22 accept
                limit rate 5/second burst 5 packets log prefix "FW-INPUT-DROP: "
        }

        chain forward {
                type filter hook forward priority filter; policy drop;
                ct state established,related accept
                ct state invalid drop
                iifname "enp0s8" oifname "enp0s10" ip saddr 10.10.10.0/24 ip daddr 10.10.30.10 tcp dpo
rt { 1883, 8883 } accept
                iifname "enp0s9" accept
                limit rate 5/second burst 5 packets log prefix "FW-FWD-DROP: "
        }
}
vboxuser@gw:~$
```

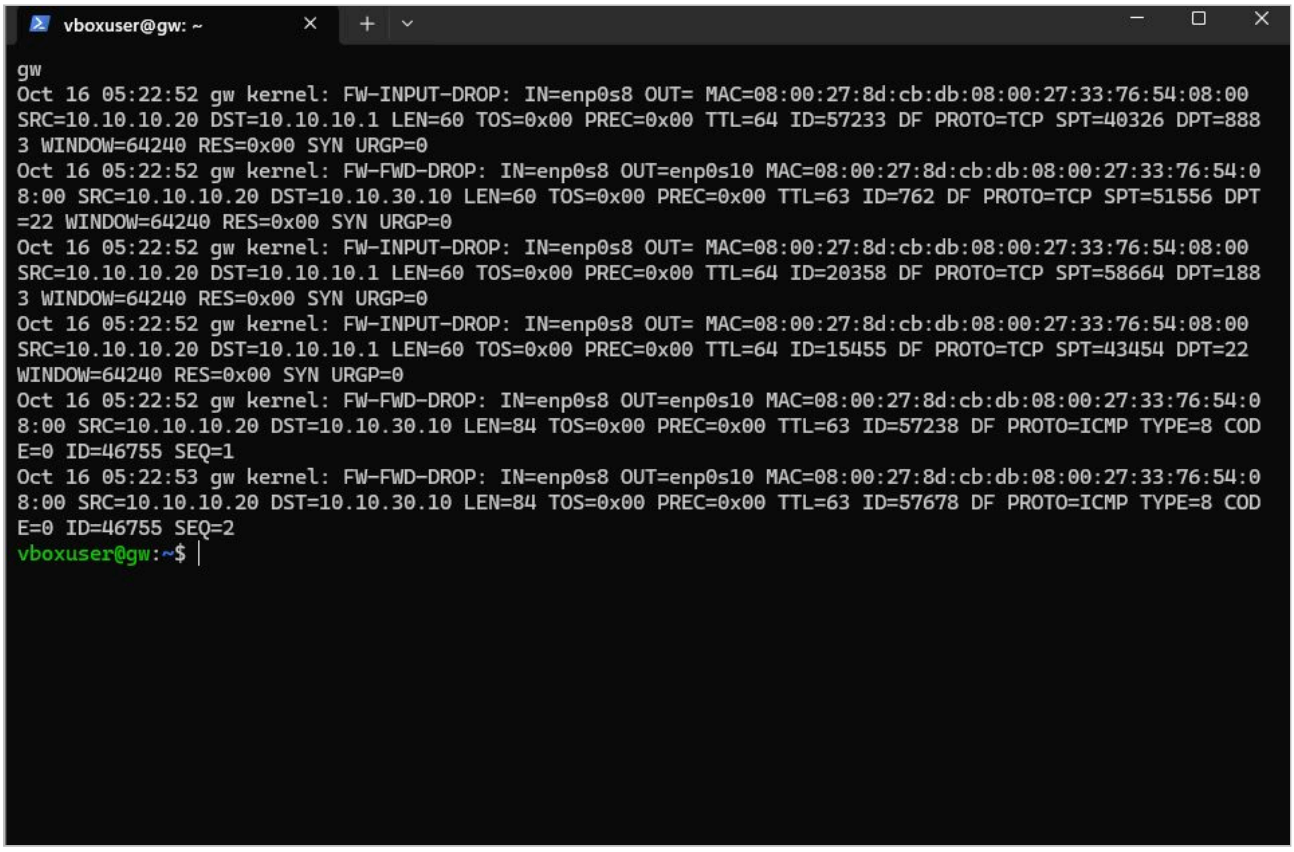
nftables iotfw ruleset

Same nmap scan as the baseline, run again after the firewall was in place:



nmap after the firewall

The firewall also records what it blocks:

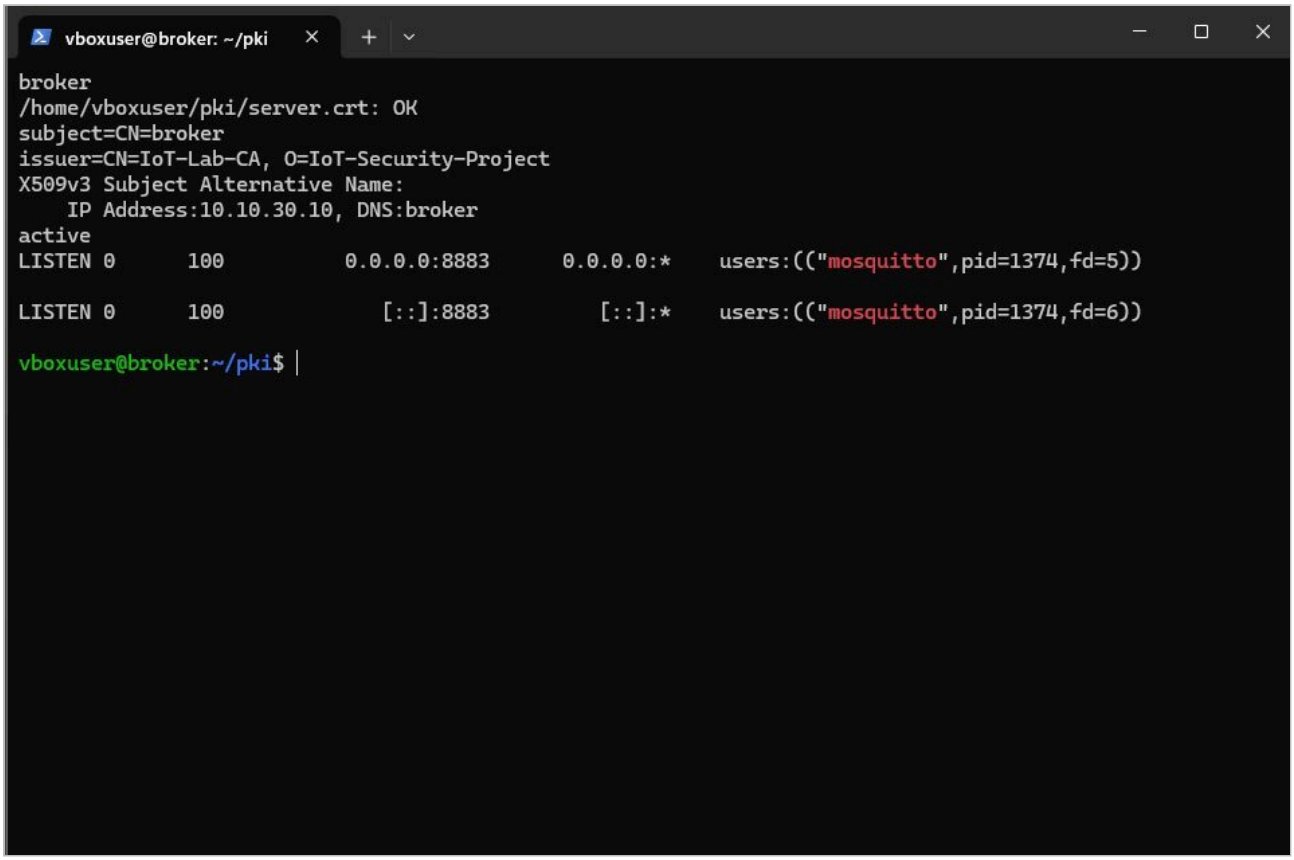


firewall drop log entries

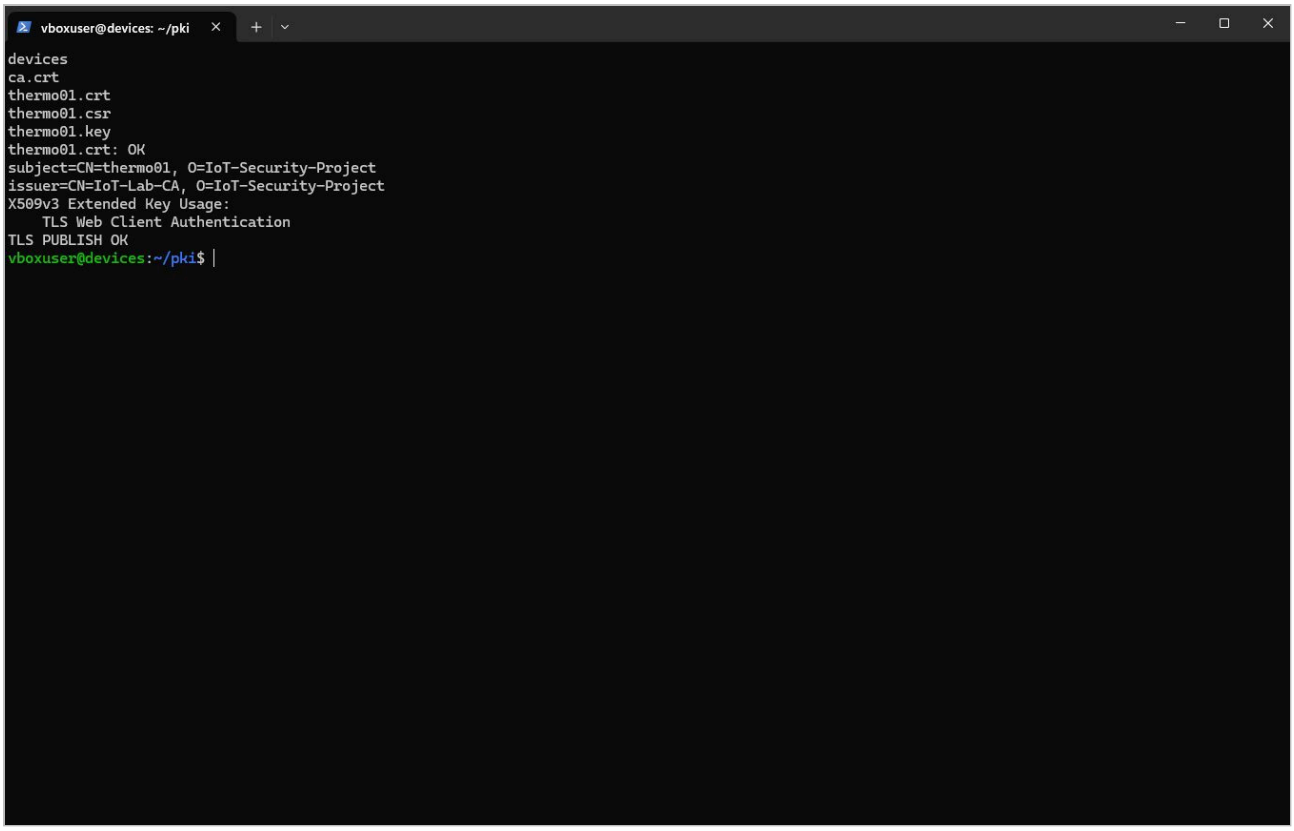
TLS and mutual authentication

The broker now accepts only TLS on port 8883, and only clients that present a certificate signed by my own CA. I created the CA (IoT-Lab-CA), a server certificate for the broker, and one client certificate each for thermo01 and admin01.

The server certificate has CN=broker and a SAN of 10.10.30.10, so clients can verify the broker by its IP.

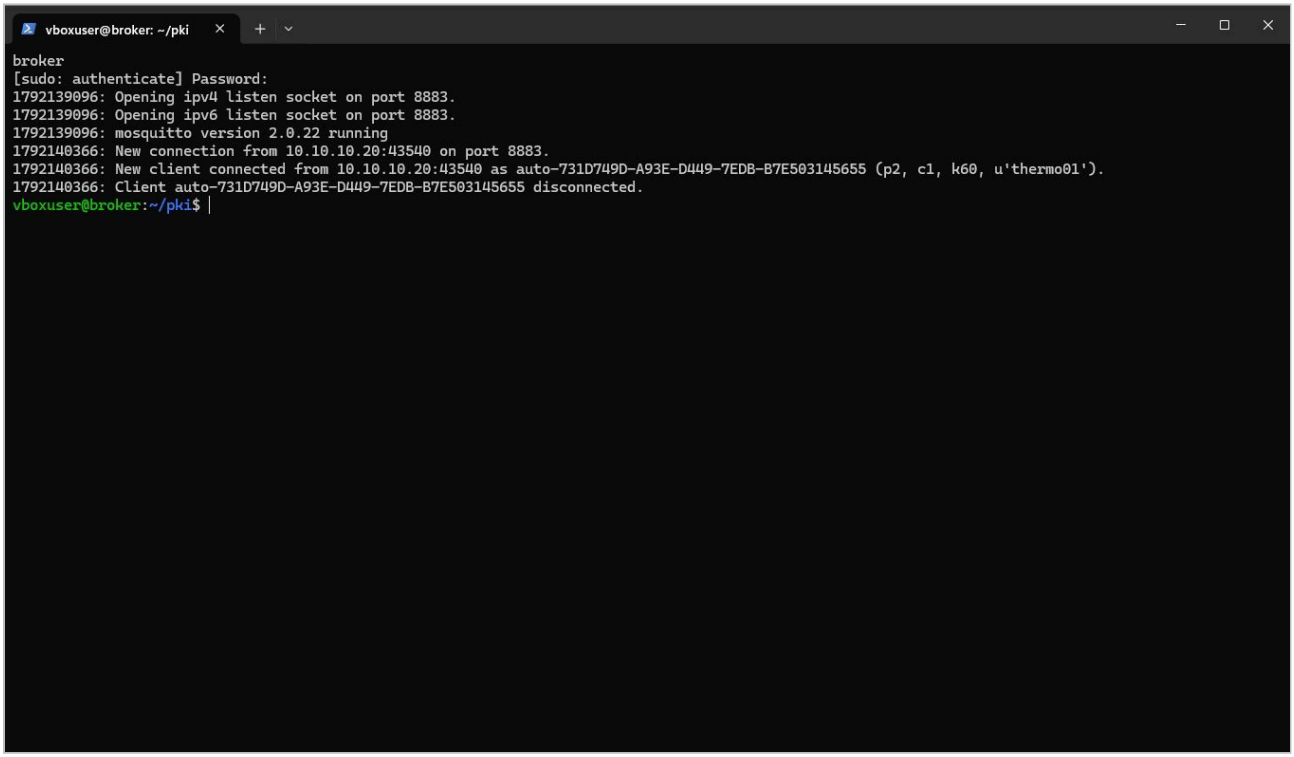


broker server certificate



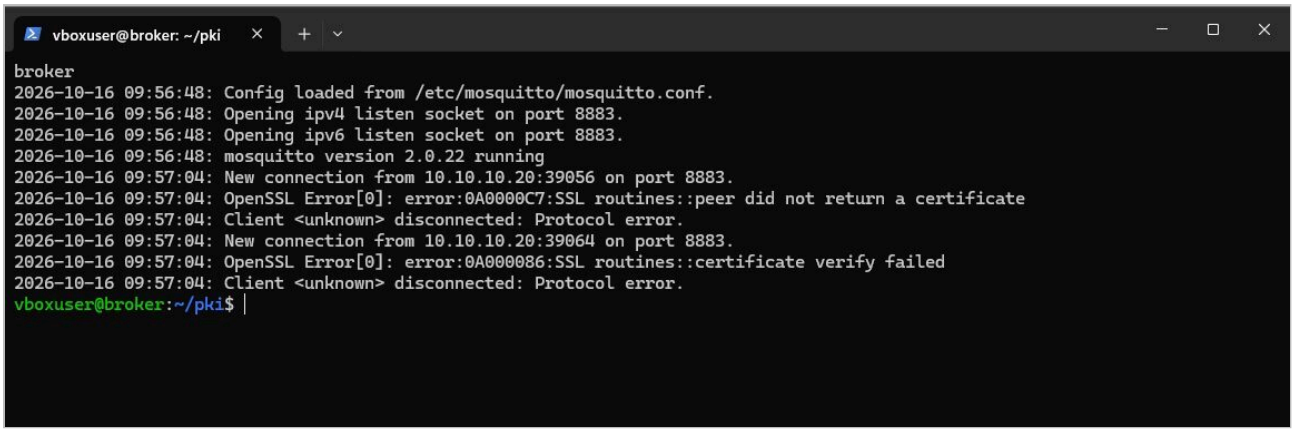
client certificate

The broker log shows the TLS connections arriving. Its timestamps are epoch seconds, not dates.



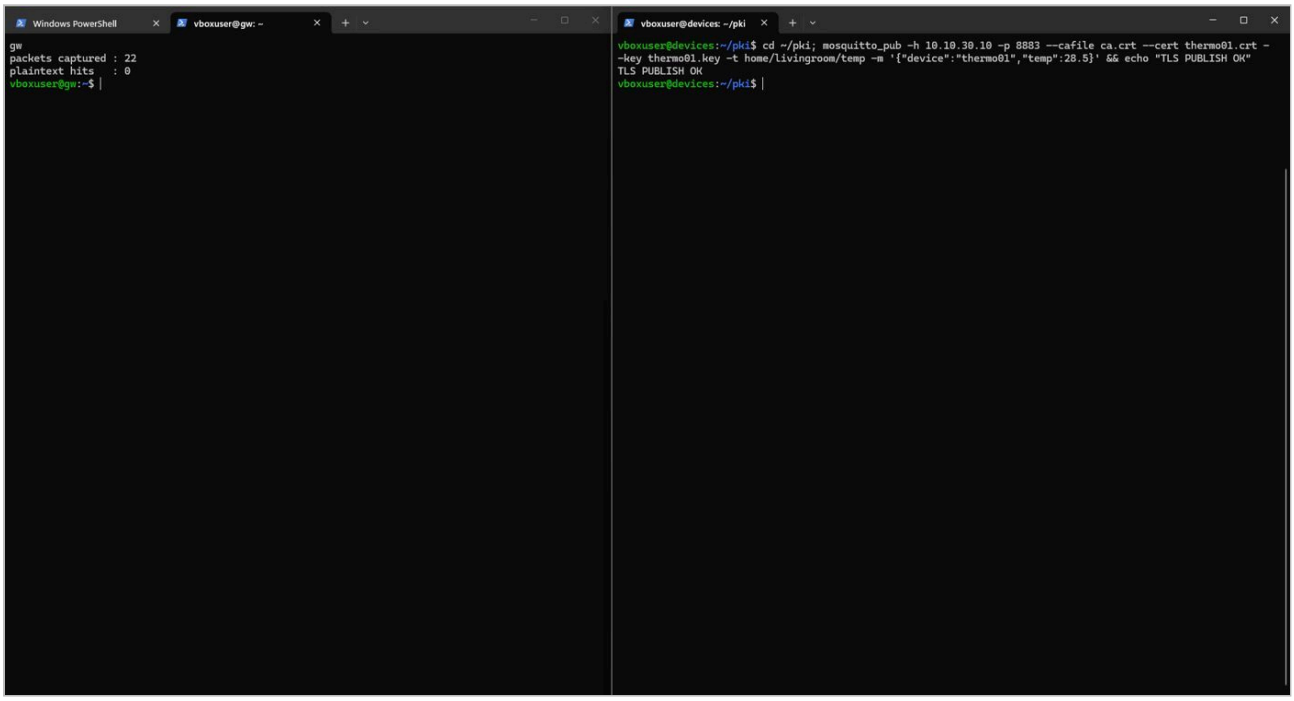
Mosquitto log with TLS connections

Negative tests. A client with no certificate is rejected. A client with a certificate from a different CA is also rejected.



no-cert and wrong-CA connections rejected

Capture test. I repeated the tcpdump from the baseline. This time the plaintext search returned 0 hits, so the payloads are encrypted on the wire.



encrypted capture, plaintext hits 0

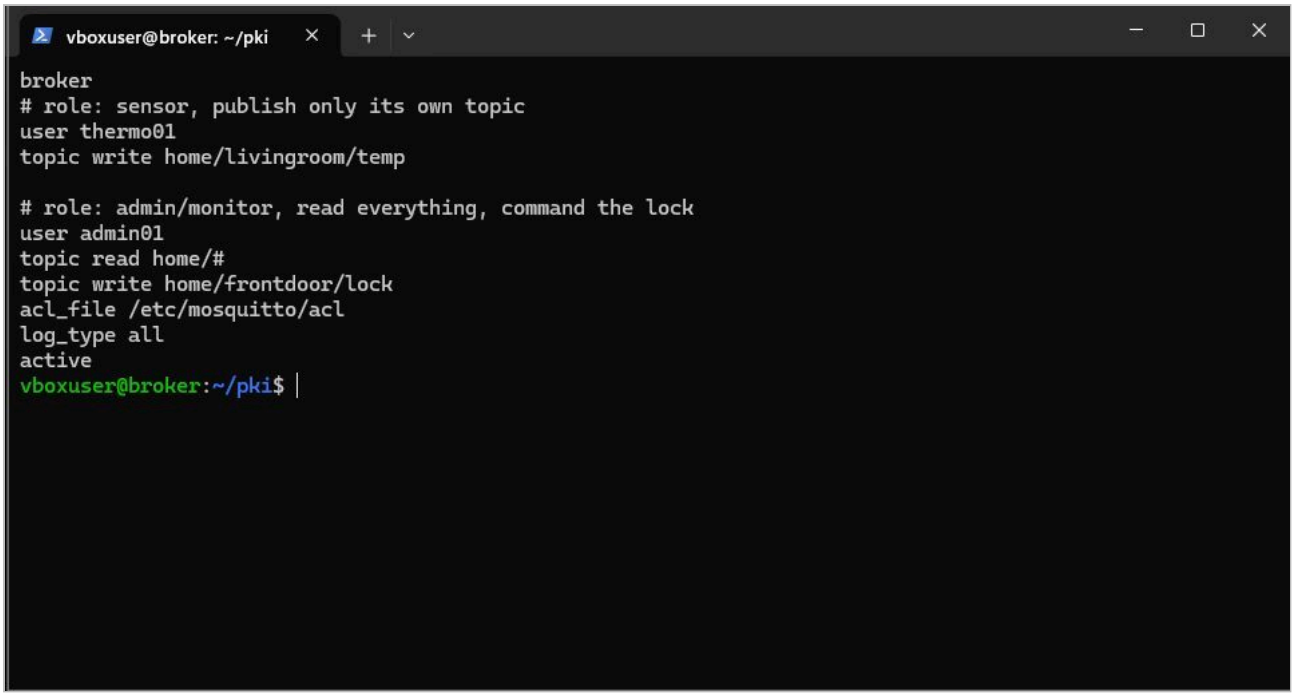
Authorization (ACL)

A valid certificate proves who you are, not what you may do. The Mosquitto ACL file limits each identity to the topics it needs.

Identity	Can write	Can read
thermo01	home/livingroom/temp	not granted
admin01	home/frontdoor/lock	home/#

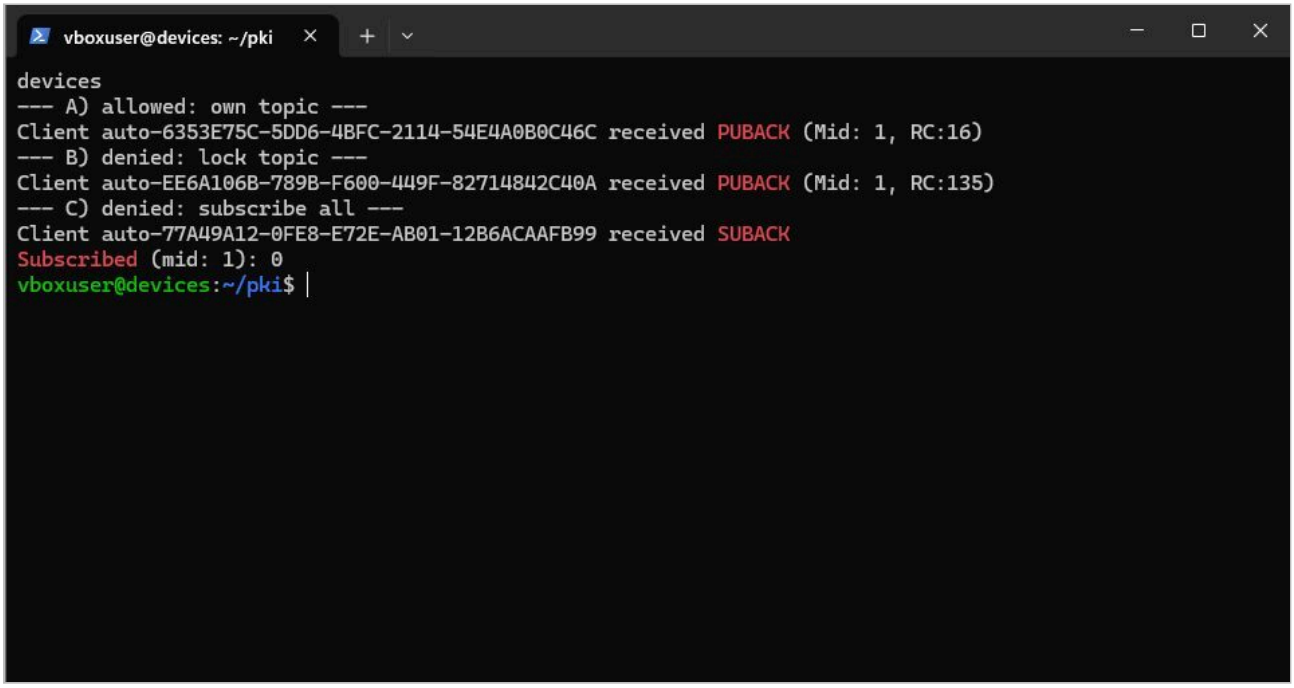
This is role-based access control with two roles. The **device** role (thermo01) can only publish its own reading. The **admin** role (admin01) can read everything and operate the lock. Adding a new device means

giving it a certificate and one ACL entry, not editing the broker's code.



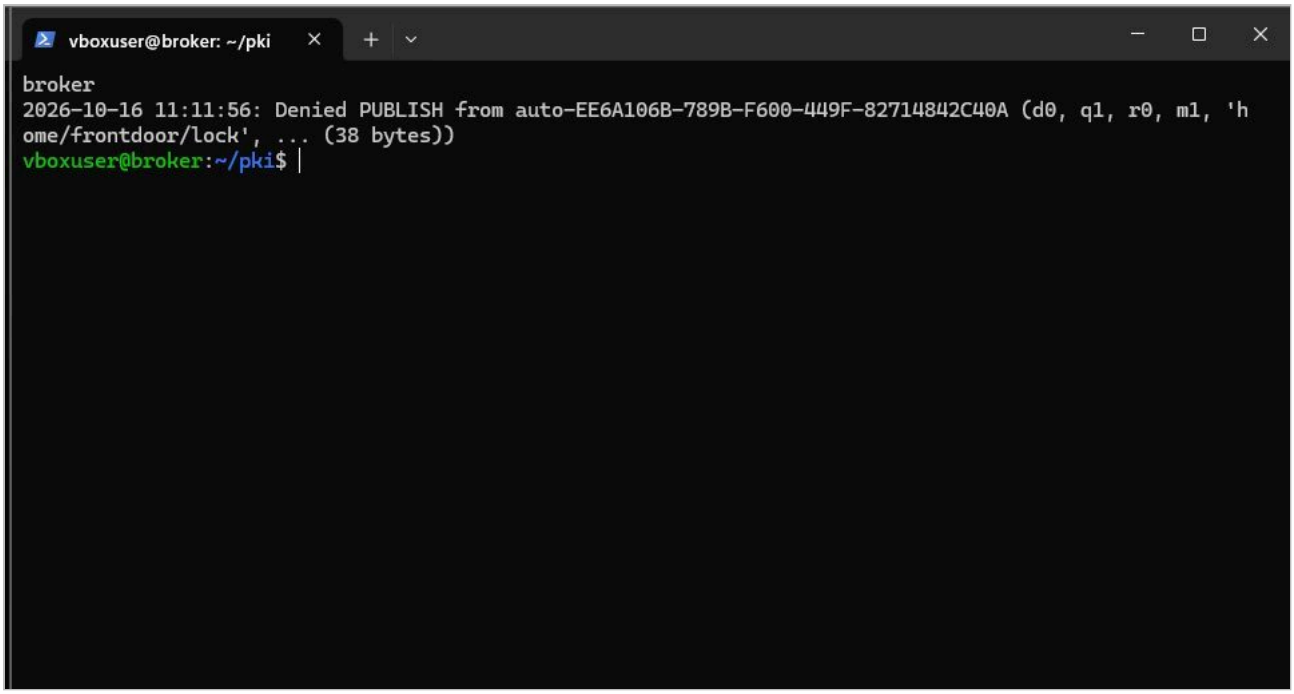
ACL file

Allowed and denied tests. thermo01 can publish to its own topic. Its attempt to publish to the door lock topic is denied.



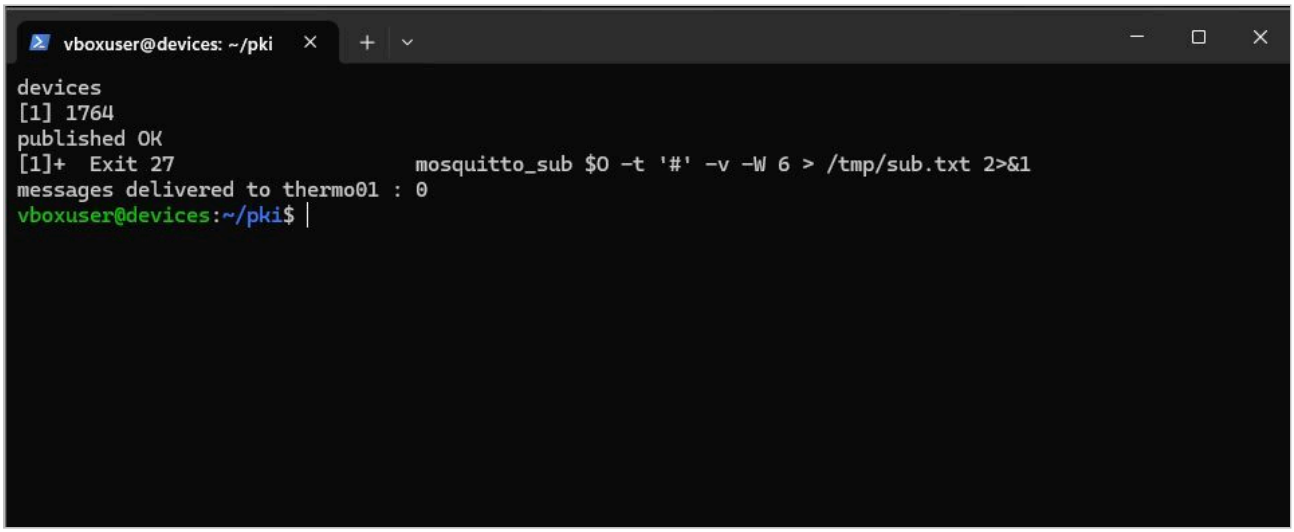
ACL allowed and denied tests

The broker logs the denied publish:



broker log of the denied publish

Read this pair together. Mosquitto returns a SUBACK even when the ACL denies a subscription, so the first screenshot alone can look like a success. The proof is the subscriber receiving nothing: "messages delivered: 0".



subscriber received 0 messages

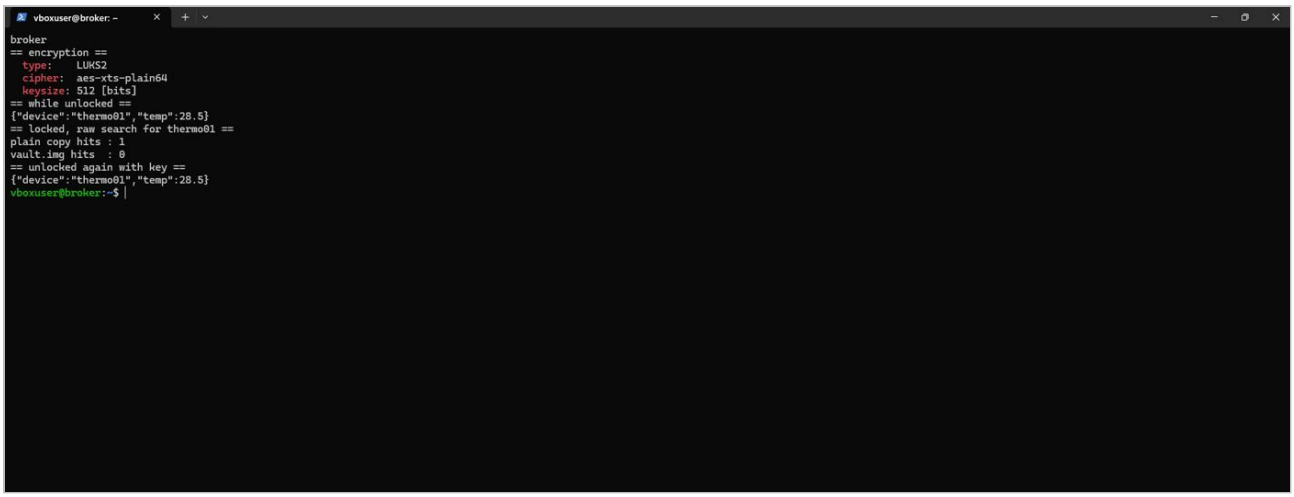
Encryption at rest

TLS protects data on the wire. I also protected it on disk, in case someone gets hold of a VM or a device.

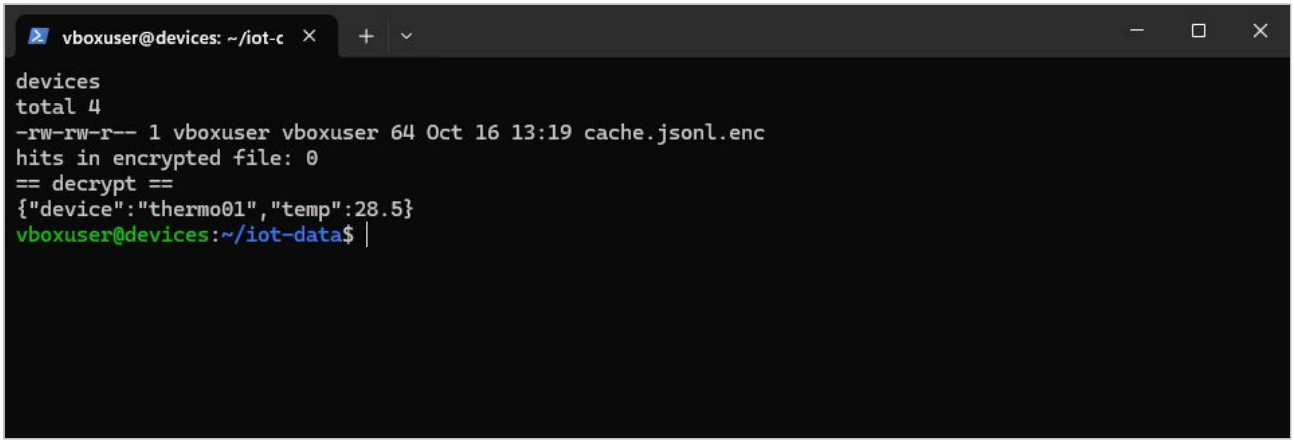
- **Broker:** a LUKS2 encrypted vault, cipher aes-xts-plain64 with a 512-bit key.
- **Device:** the local cache is stored as cache.jsonl.enc, not as readable JSON.

aes-xts-plain64 with a 512-bit key is AES-256, because XTS mode uses two 256-bit keys. The broker is the stand-in for cloud storage in this lab.

The vault screenshot includes a line "plain copy hits : 1". That is the control file I wrote before encrypting, which shows the test string was readable before the data went into the vault.



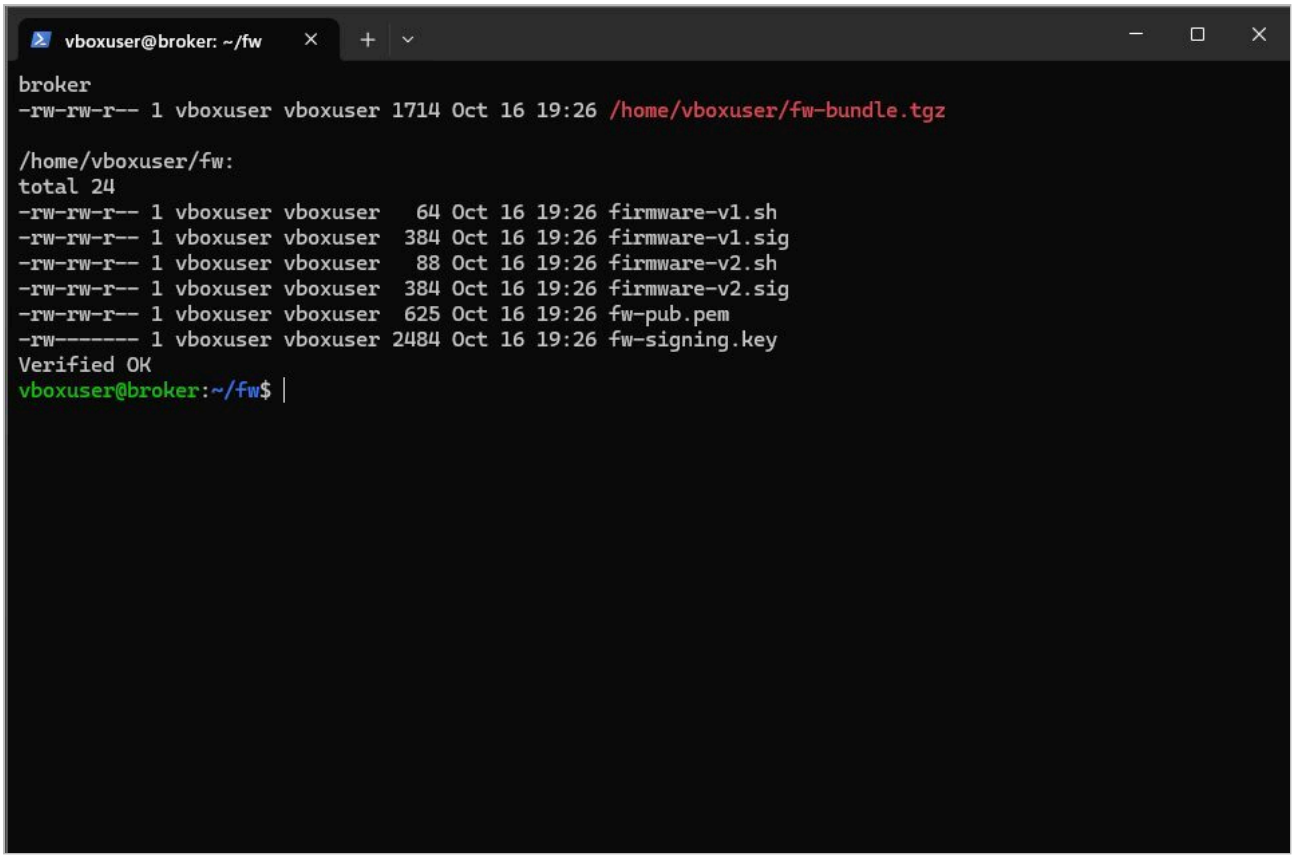
LUKS2 vault on the broker



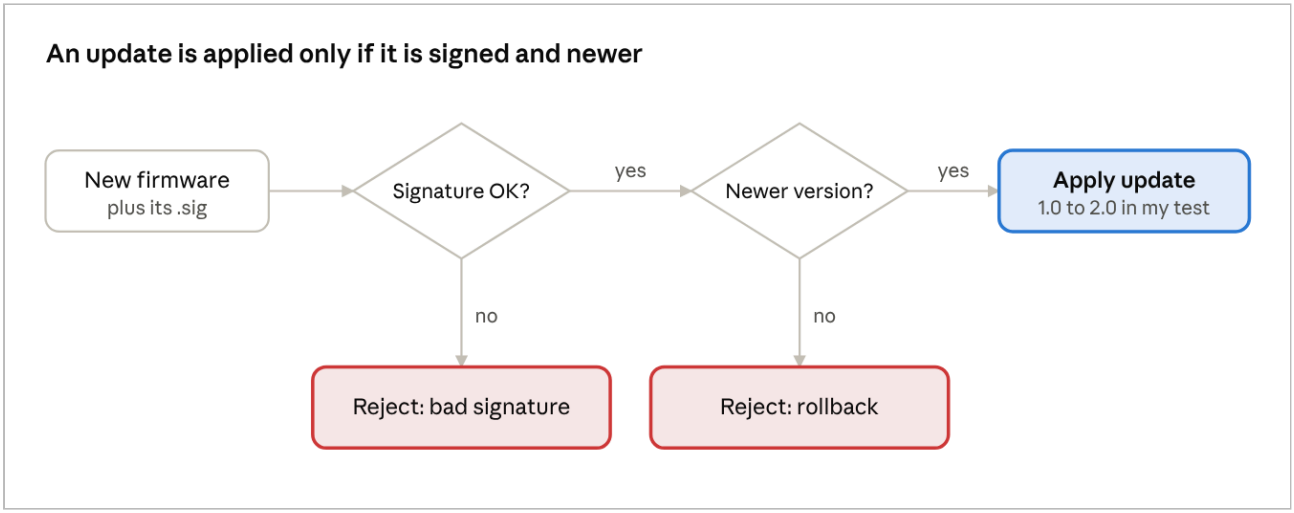
encrypted cache on the device

Firmware signing and secure updates

Firmware is the code a device trusts most, so it must be proved genuine before it runs. I signed each firmware version with an OpenSSL key. The private signing key has permissions 600; the device holds only the public key.

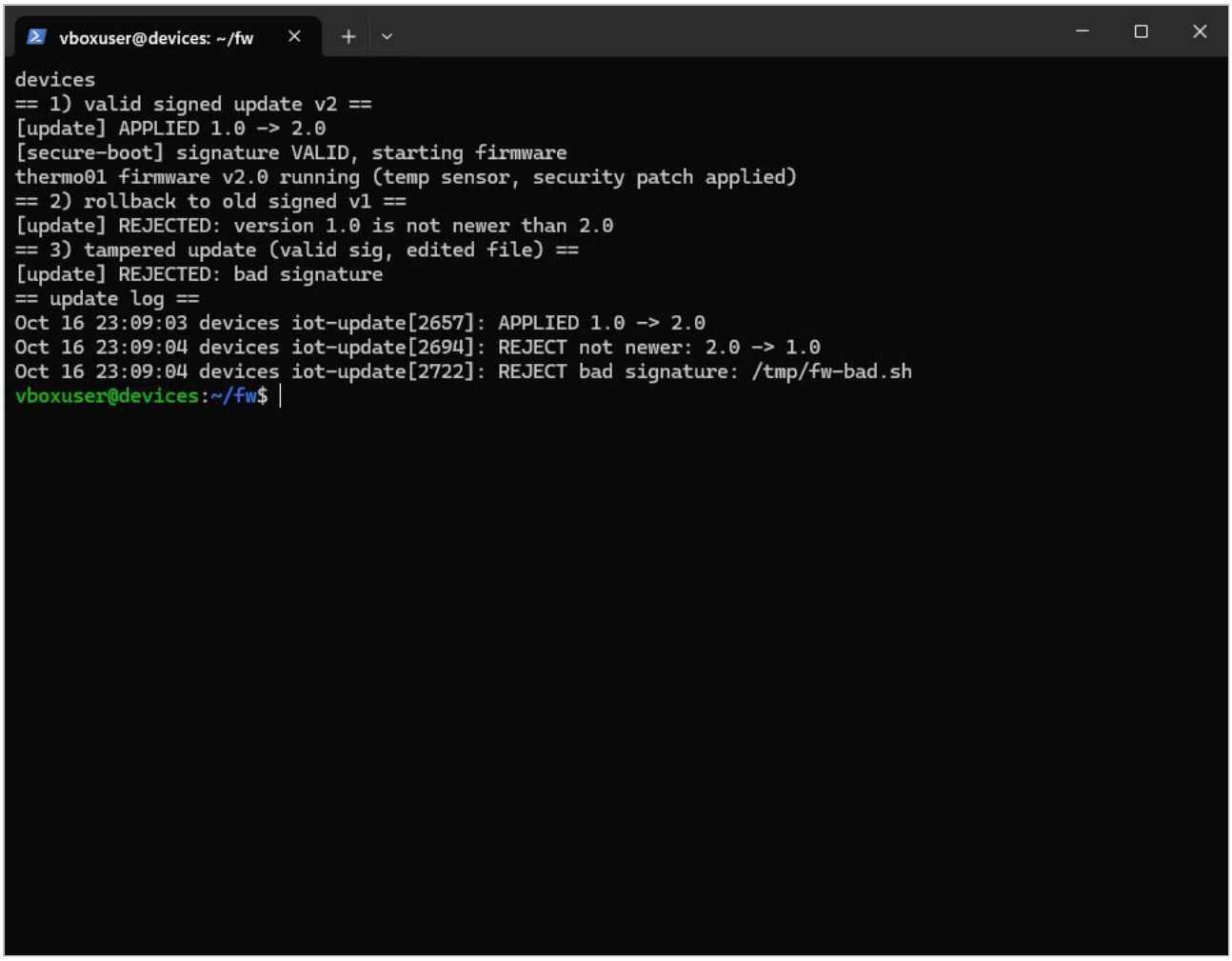


firmware signatures verified OK



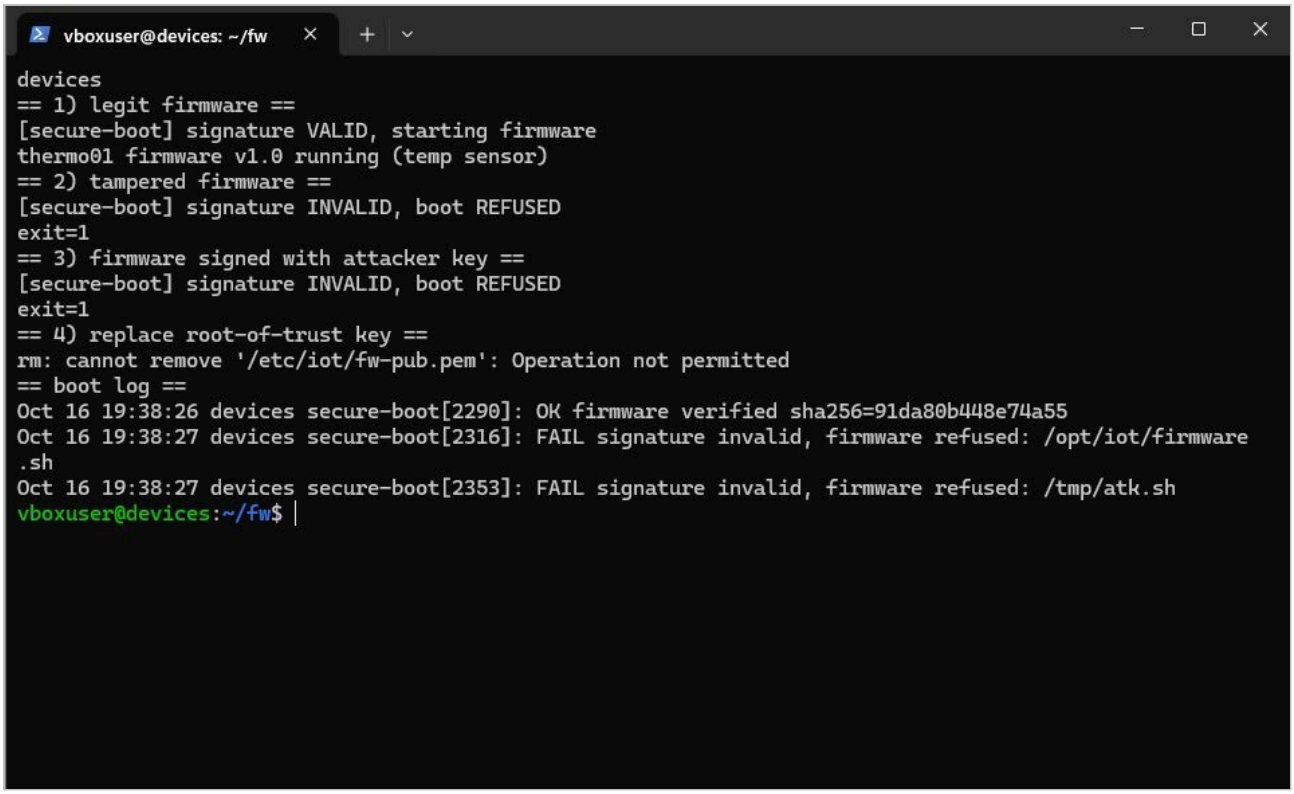
firmware update decision ·2 checks

The update client applied 1.0 to 2.0. It rejected a tampered file, a bad signature and a rollback to the older version.



update applied, rollback and tampered file rejected

Simulated secure boot. A script checks the signature at start and refuses tampered firmware or firmware signed by the attacker's key. The root-of-trust public key is marked immutable, so it cannot be swapped.

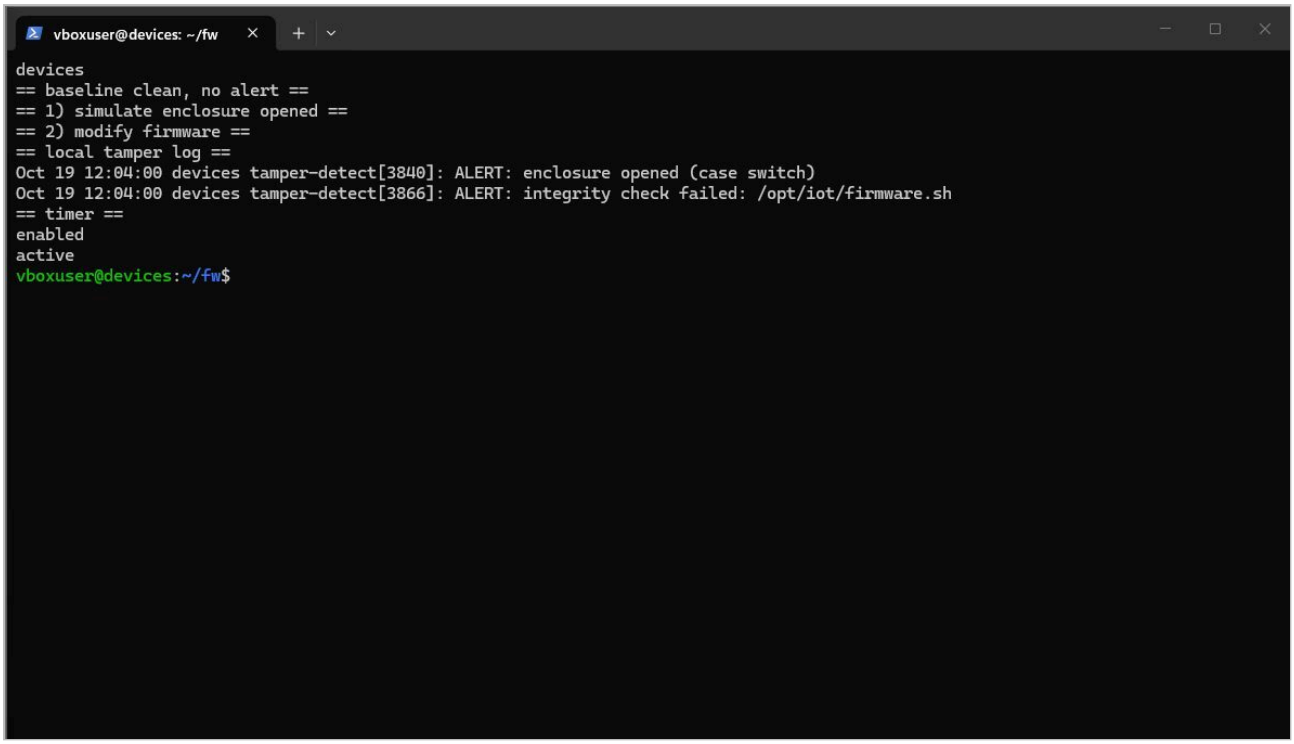


simulated secure boot refusing bad firmware

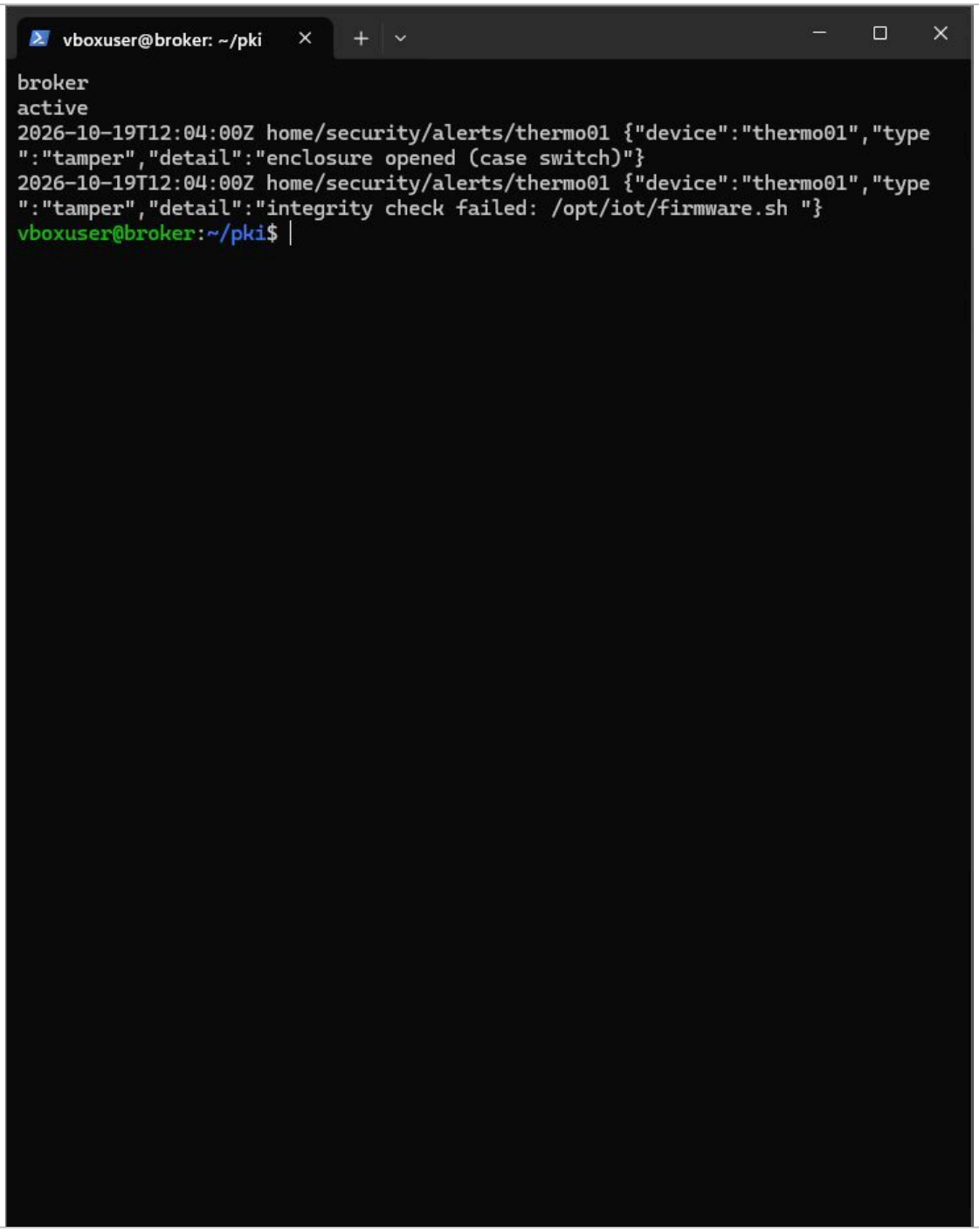
Physical security and tamper detection

Someone with physical access can open a device, swap its firmware or pull its storage. A VM has no case, so I simulated the two signals a real device would raise: a case switch and a firmware integrity check.

When either trips, the device publishes an alert over the same TLS connection to home/security/alerts/thermo01 on the broker.



simulated tamper detection alert



tamper alert received on the broker

This is a software simulation. A real product needs a hardware tamper switch, a sealed enclosure and secure storage for keys, none of which a VM can show. The data-at-rest controls above cover the stolen-storage case in software.

Secure hardware specification

I could not test hardware in a VM, so this is the specification I would require from a real device:

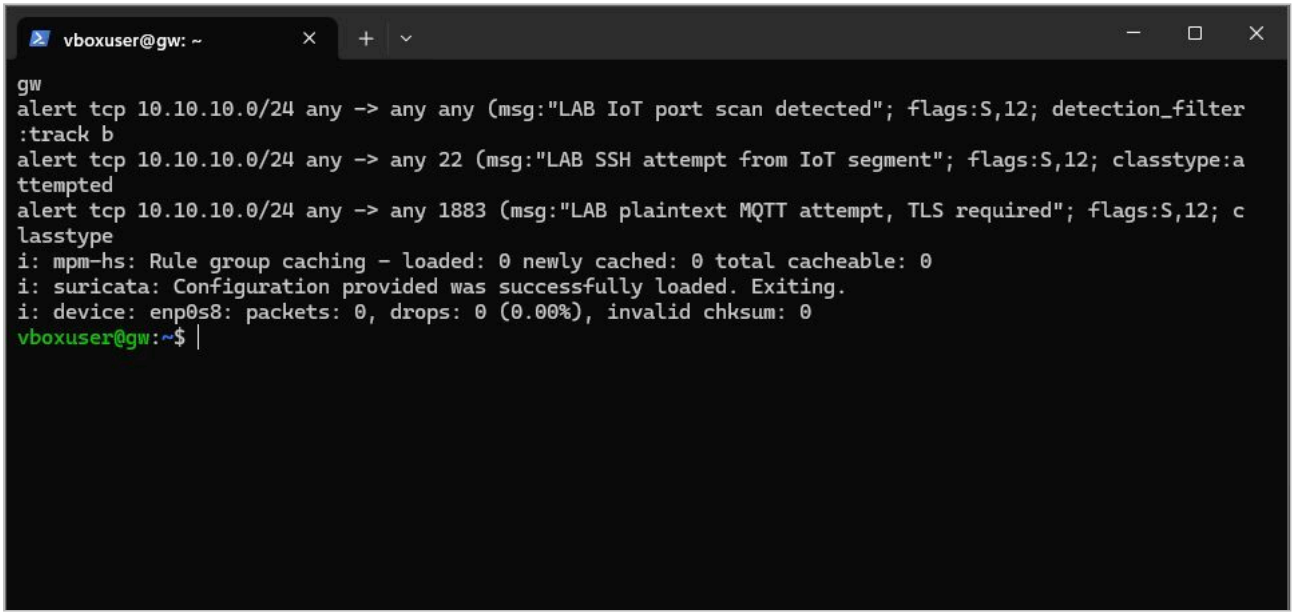
- A secure element or TPM that holds the device's private key and never exports it.
- A boot ROM that verifies the first-stage firmware, so the chain of trust starts in hardware.

- Debug interfaces (JTAG, UART) disabled or locked on production units.
- A tamper-evident enclosure with a switch wired to the tamper alert.
- Flash encryption, so a dumped chip does not reveal firmware or data.

Monitoring and IDS

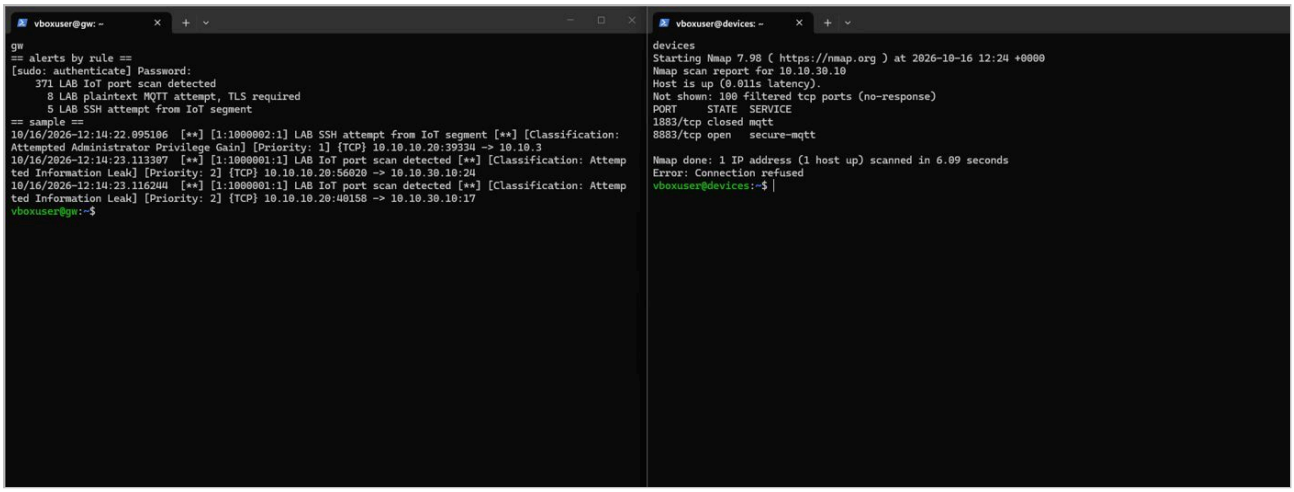
Prevention is not enough, so I added detection. Suricata runs on the gateway, watching the iot-net interface (enp0s8) with three custom rules.

sid	Detects	Alerts seen
1000001	Port scan from the IoT segment	371
1000002	SSH from the IoT segment	5
1000003	Plaintext MQTT	8



Suricata custom rules

The alert counts come from my own scans and test traffic. The same screenshot shows a scan of devices finding 1883 closed and 8883 open, which matches the hardened config.



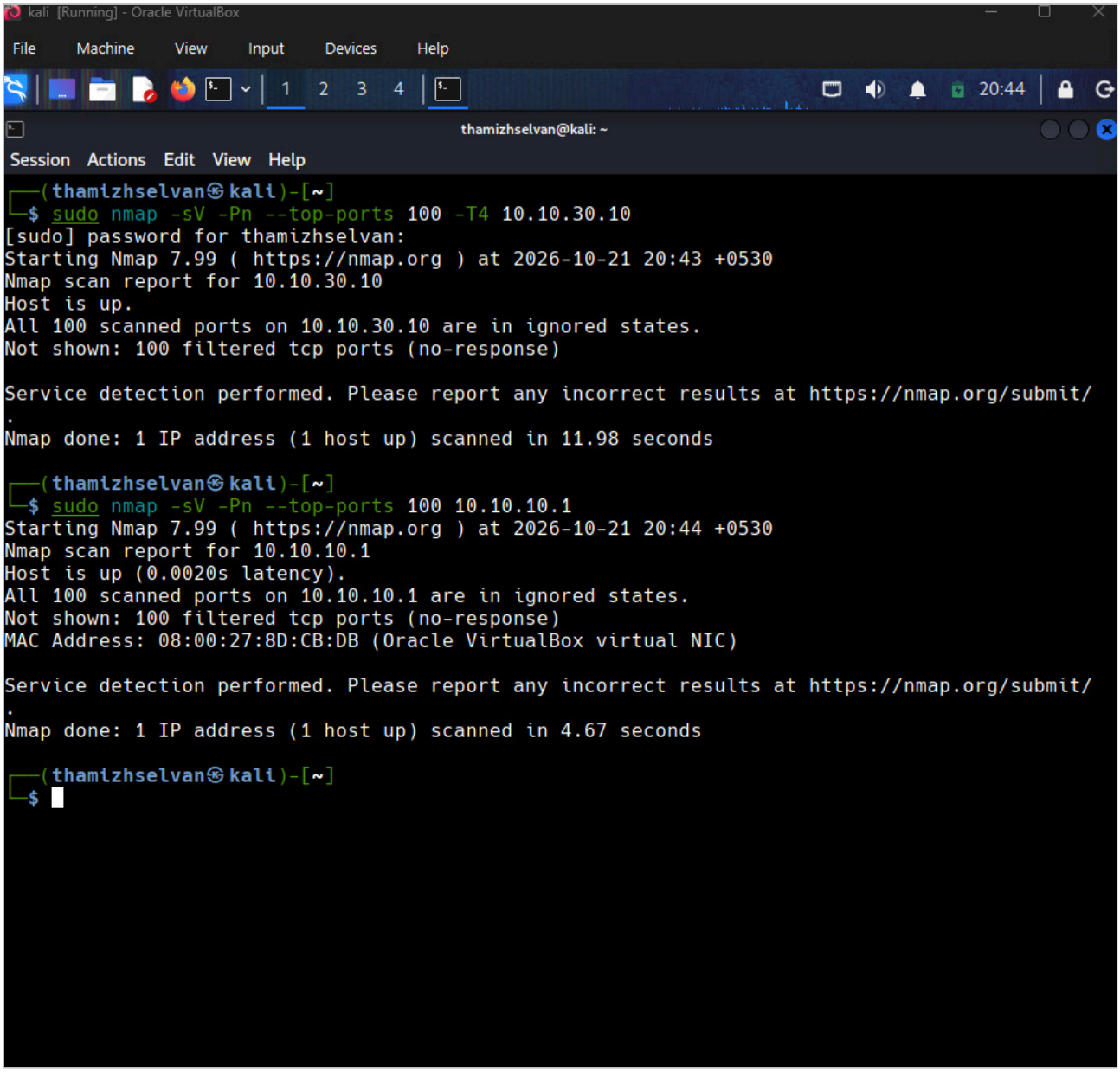
Suricata alert counts and nmap of devices

Attack tests on the hardened lab

After hardening I attacked the lab again from the Kali VM, which sits on iot-net at 10.10.10.2. The gateway at 10.10.10.1 is its only way to the other networks.

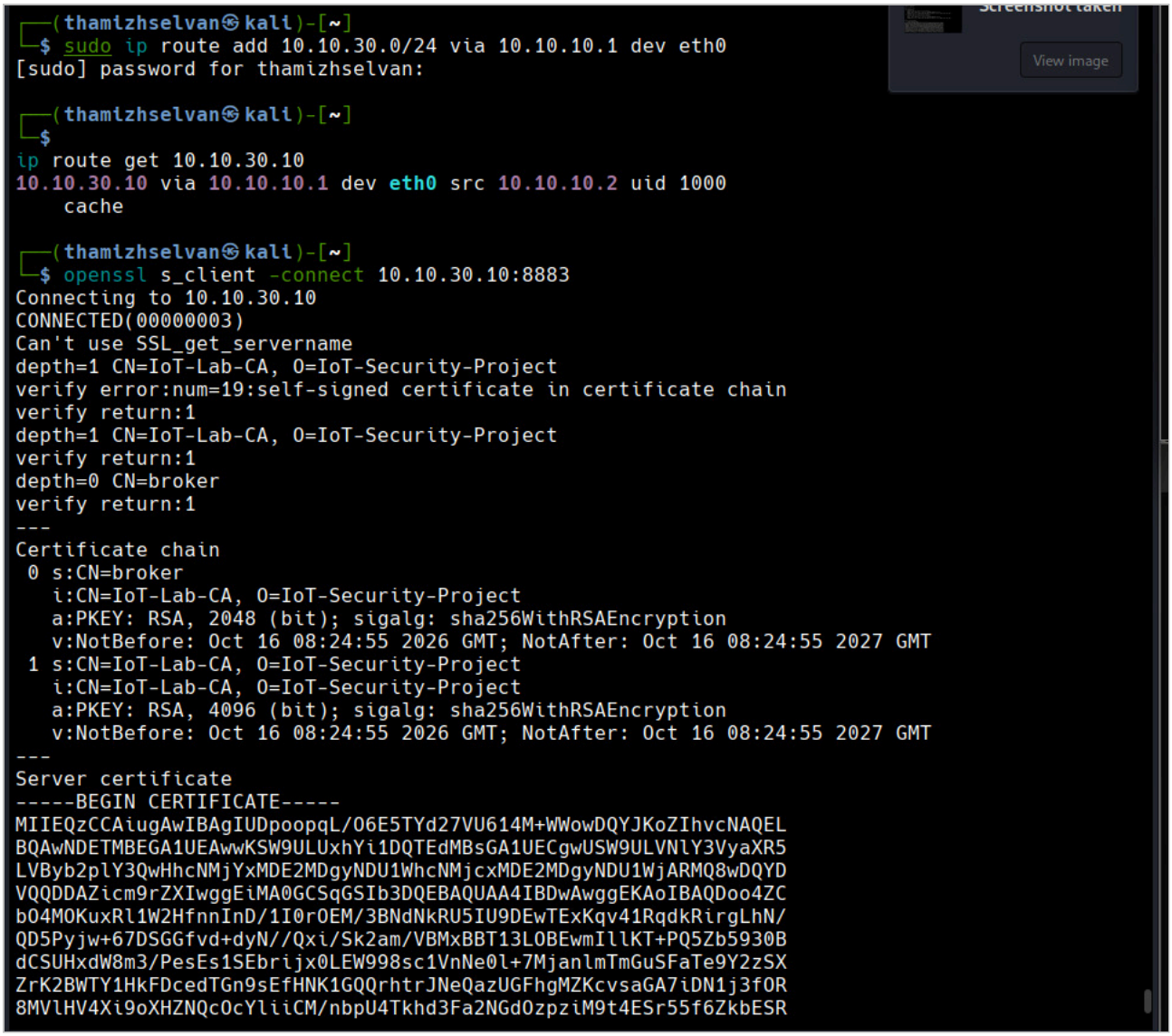
Test	What I ran	Result
Scan the gateway	nmap top 100 ports against 10.10.10.1	All 100 filtered, no response
TLS with no client certificate	openssl s_client to 10.10.30.10:8883	Handshake completes, broker alerts "certificate required" (116) and closes
TLS with a certificate from another CA	same, with a wrong-CA certificate	Broker alerts "unknown ca" (48) and closes
Sniff the TLS session	tcpdump on the capture of the 8883 session	Handshake and payload unreadable
Port scan, live	nmap -sS ports 1-1000 against the broker	Suricata rule 1000001 fires in real time

Gateway scan. Every probed port on the gateway is filtered. The input chain drops everything from iot-net.



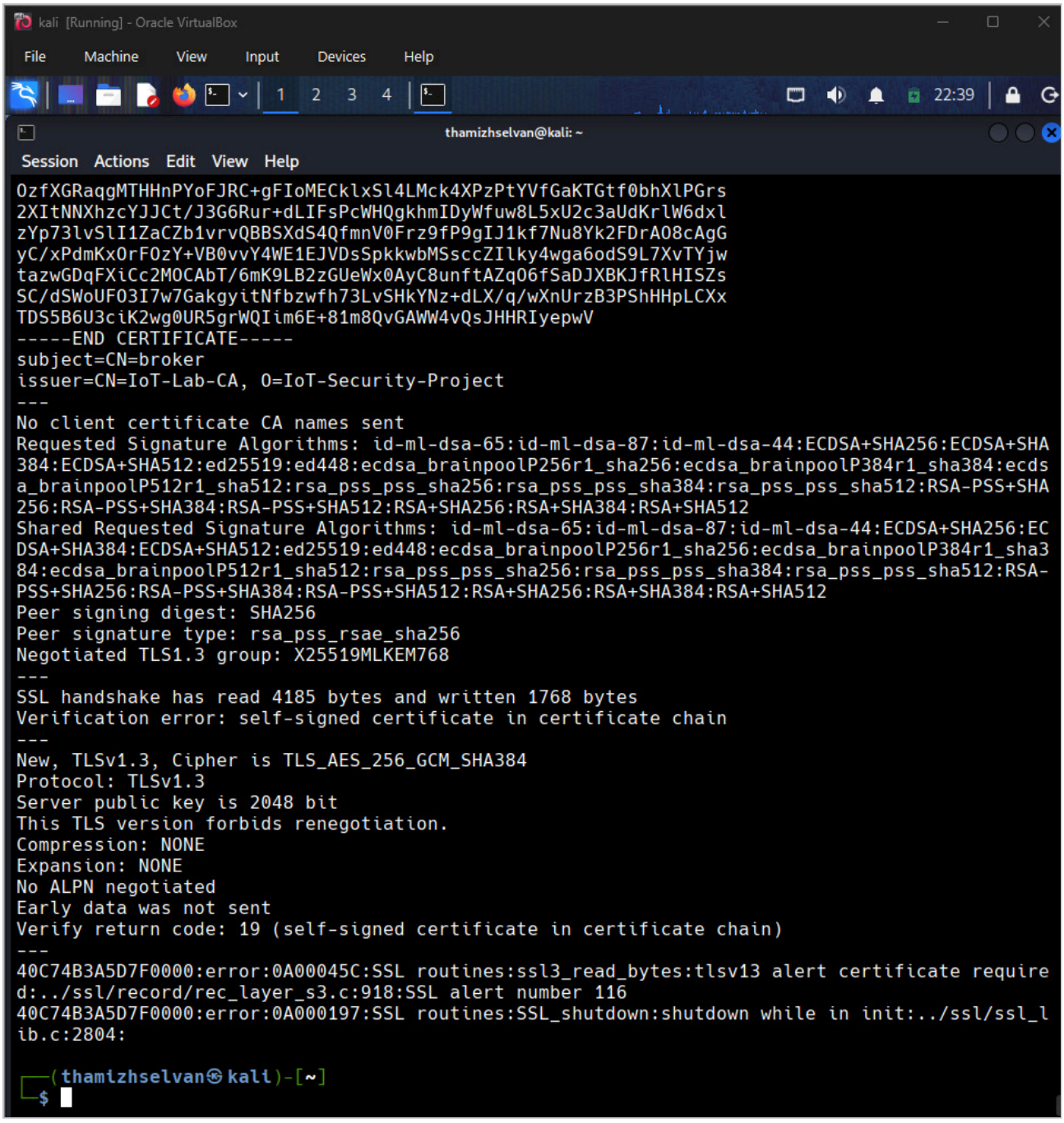
nmap top 100 against the broker and the gateway

Route to cloud-net. Kali had no route to the broker's network, so I added one through the gateway before the TLS tests.



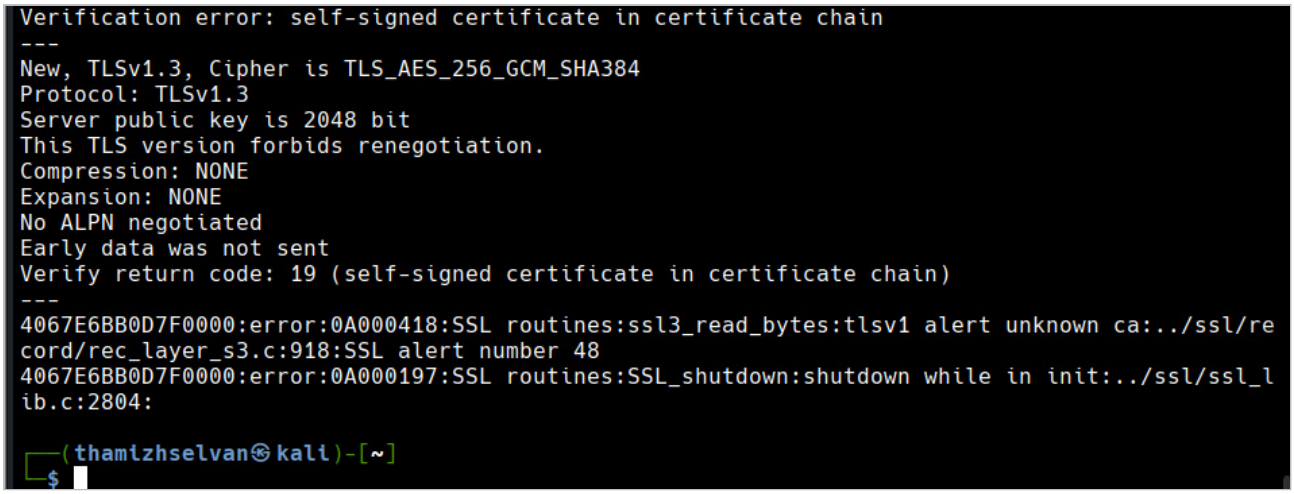
route added on Kali, TLS handshake started

No client certificate. The broker completes the TLS 1.3 handshake (TLS_AES_256_GCM_SHA384), then closes the session because no client certificate was sent. The line "self-signed certificate in chain" only means Kali does not hold IoT-Lab-CA. It is not the failure.



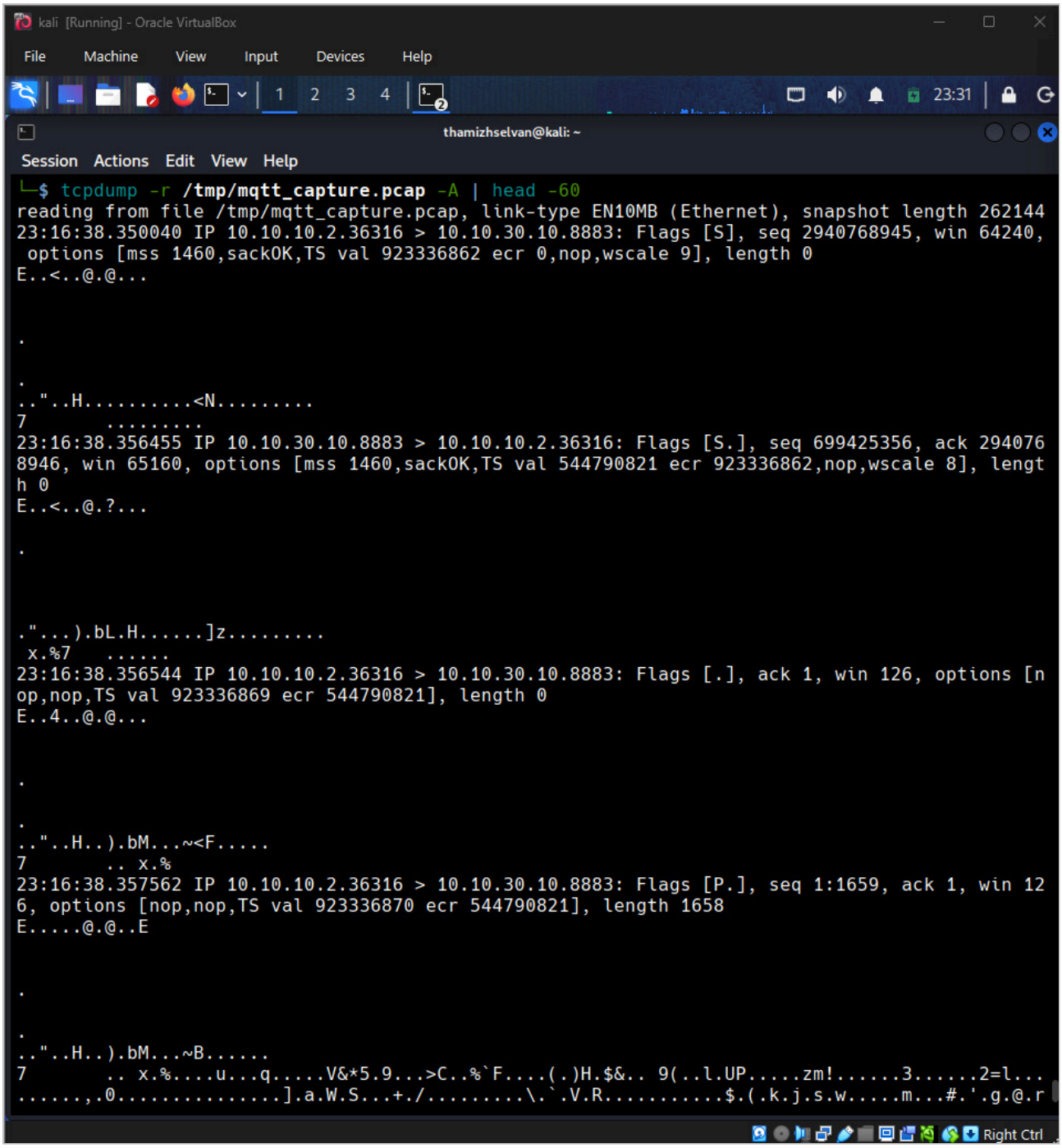
TLS without a client certificate rejected

Wrong CA.

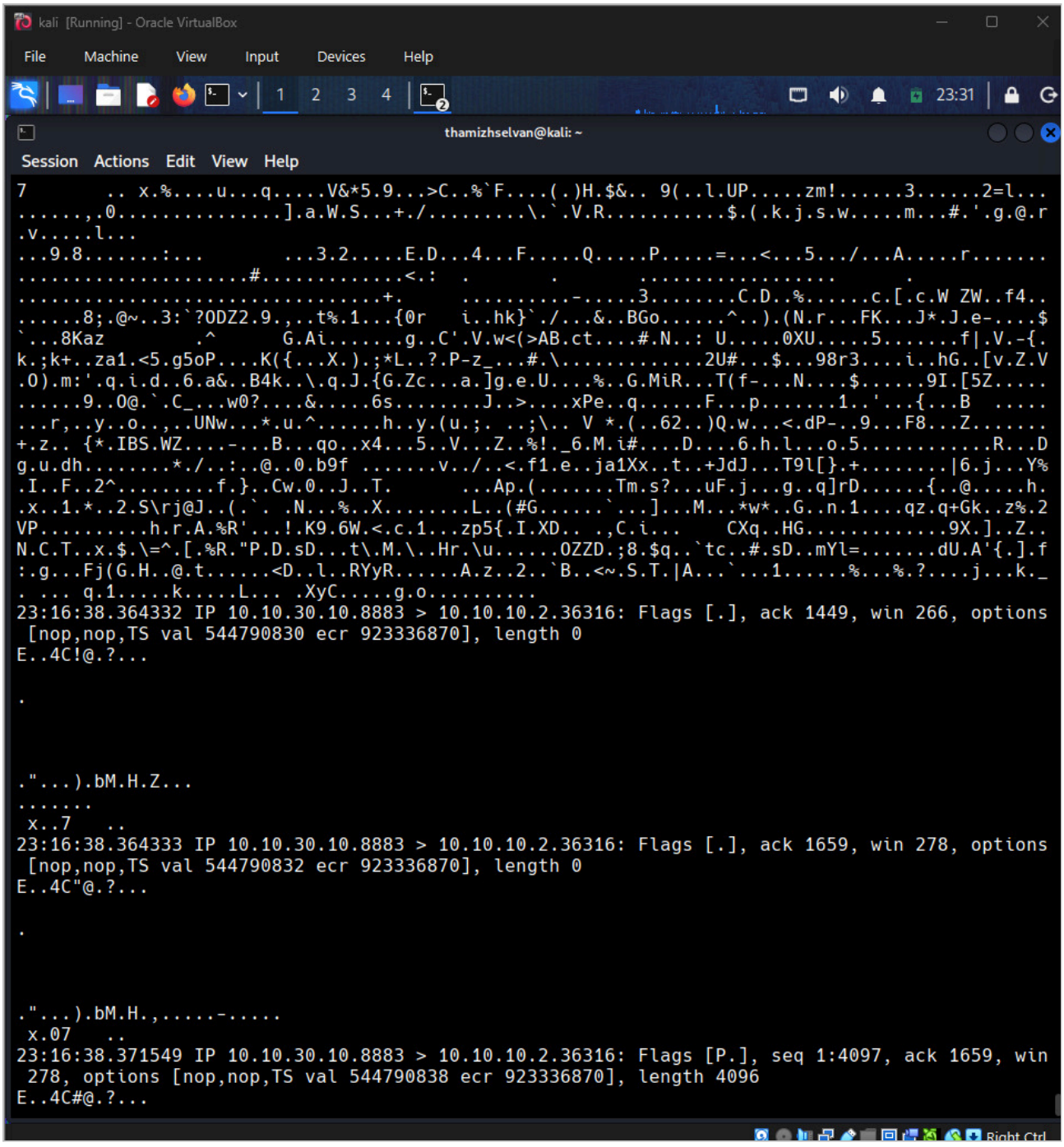


TLS with an untrusted client certificate rejected

Sniffing. The captured session shows the TCP handshake, then TLS records that read as noise. No topic names or message text appear.

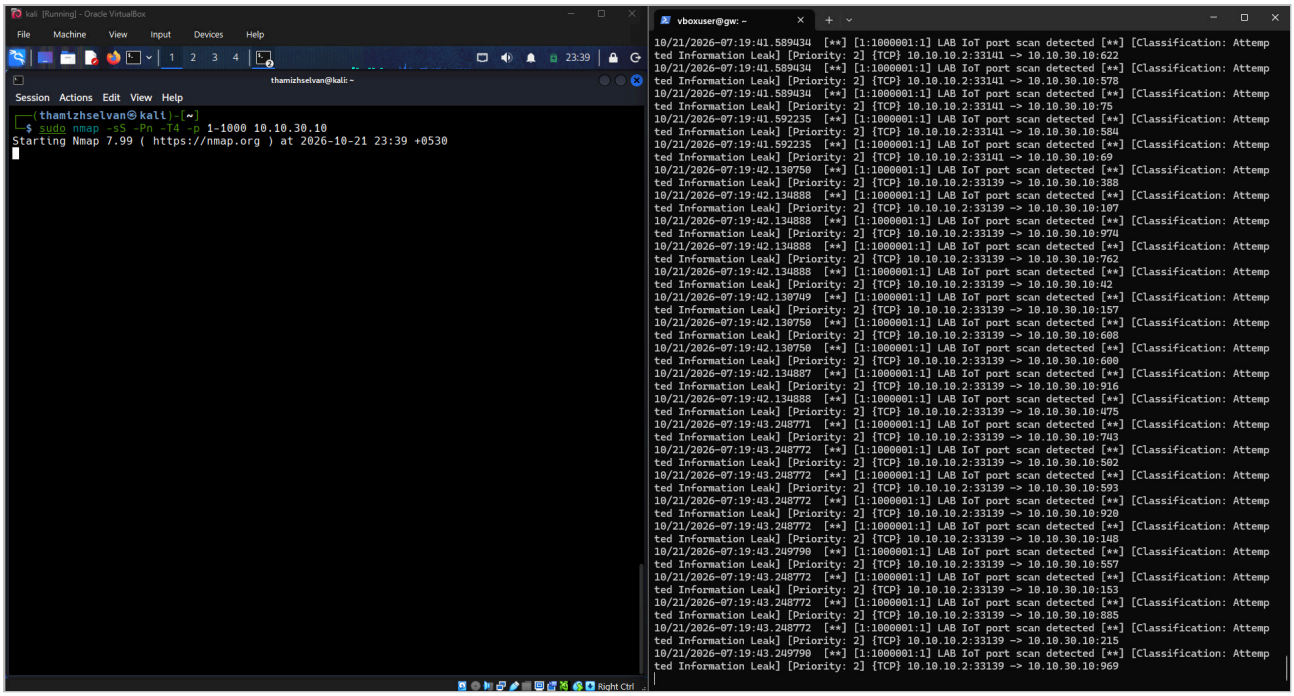


tcpdump of the TLS handshake



tcpdump payload unreadable

Detection. While the scan ran on Kali, the gateway printed Suricata alerts for each probed port.

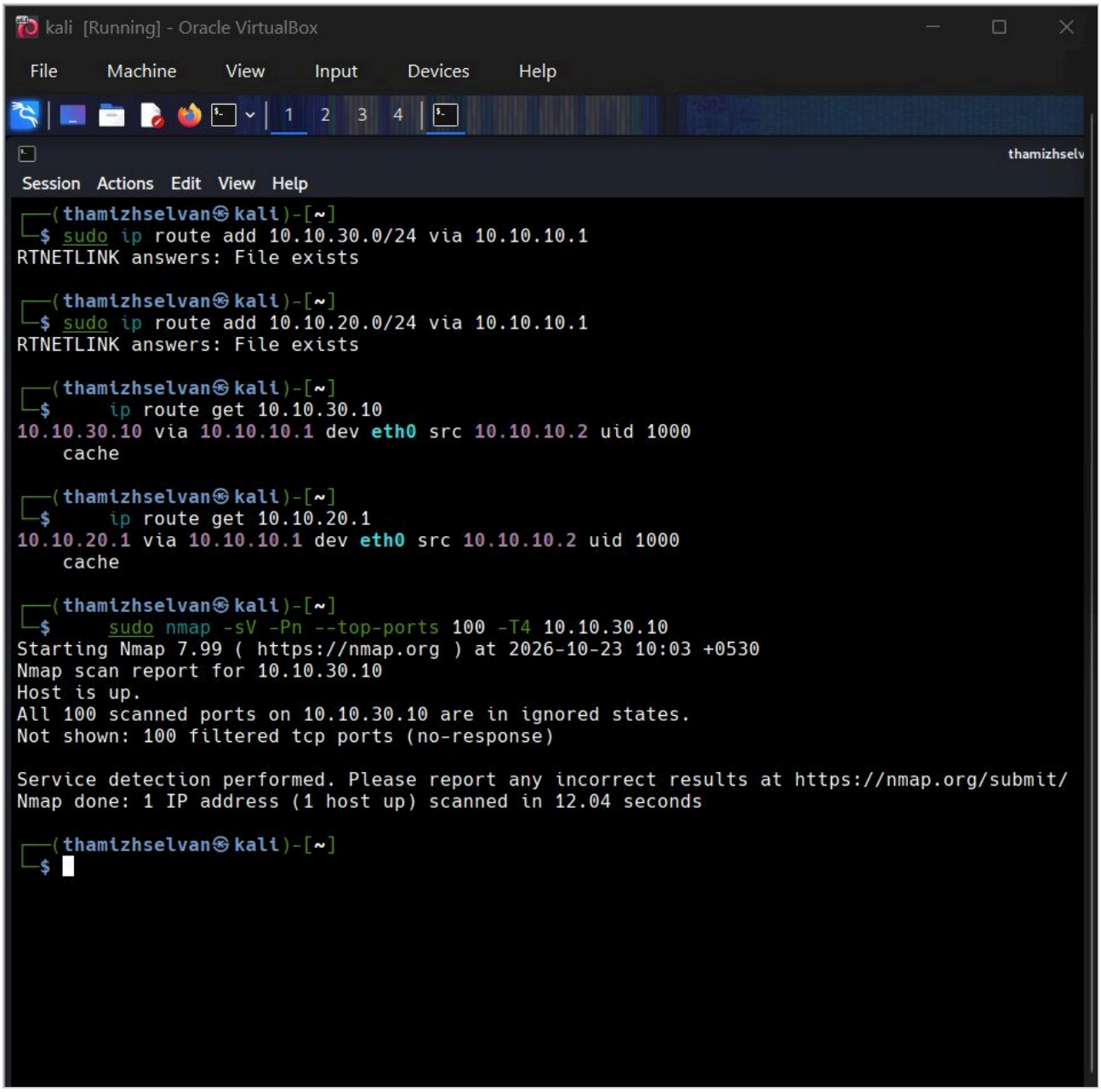


live Suricata port scan alerts on the gateway

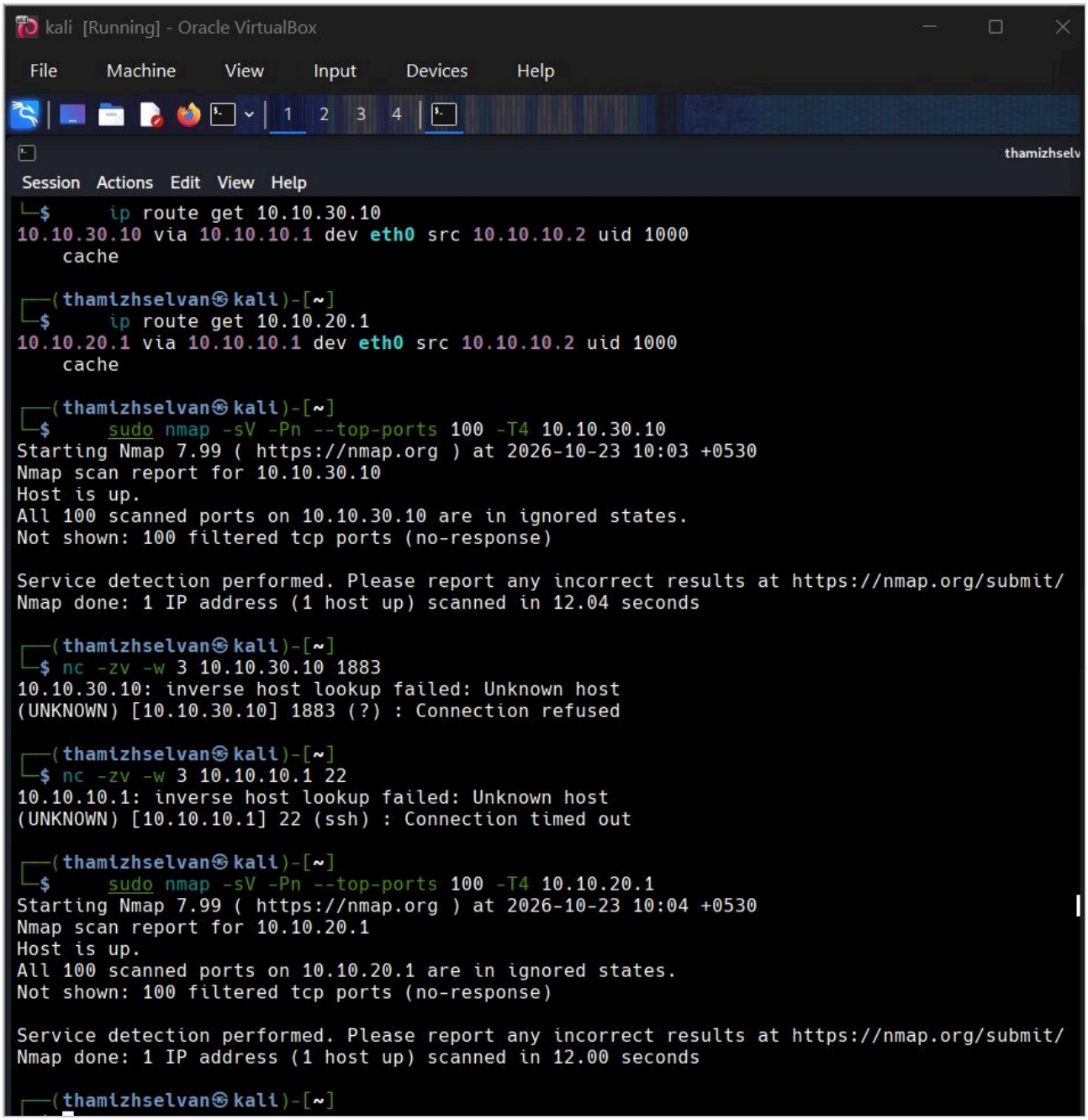
Scans with the route in place

The first scans of the broker and trusted-net ran before Kali had its route to cloud-net, so I repeated them with the routes to 10.10.30.0/24 and 10.10.20.0/24 set through the gateway (ip route get confirms both go via 10.10.10.1).

- **Broker, top 100 ports:** all 100 filtered. Port 8883 is not in nmap's top 100, so it is checked on its own.
- **Anonymous MQTT on 1883:** refused by the broker, which no longer listens on it. That showed the gateway rule still allowed 1883, so I changed it to 8883 only (below).
- **SSH to the gateway from iot-net:** times out. Only trusted-net may reach it.
- **Trusted-net gateway address, top 100 ports:** all filtered.

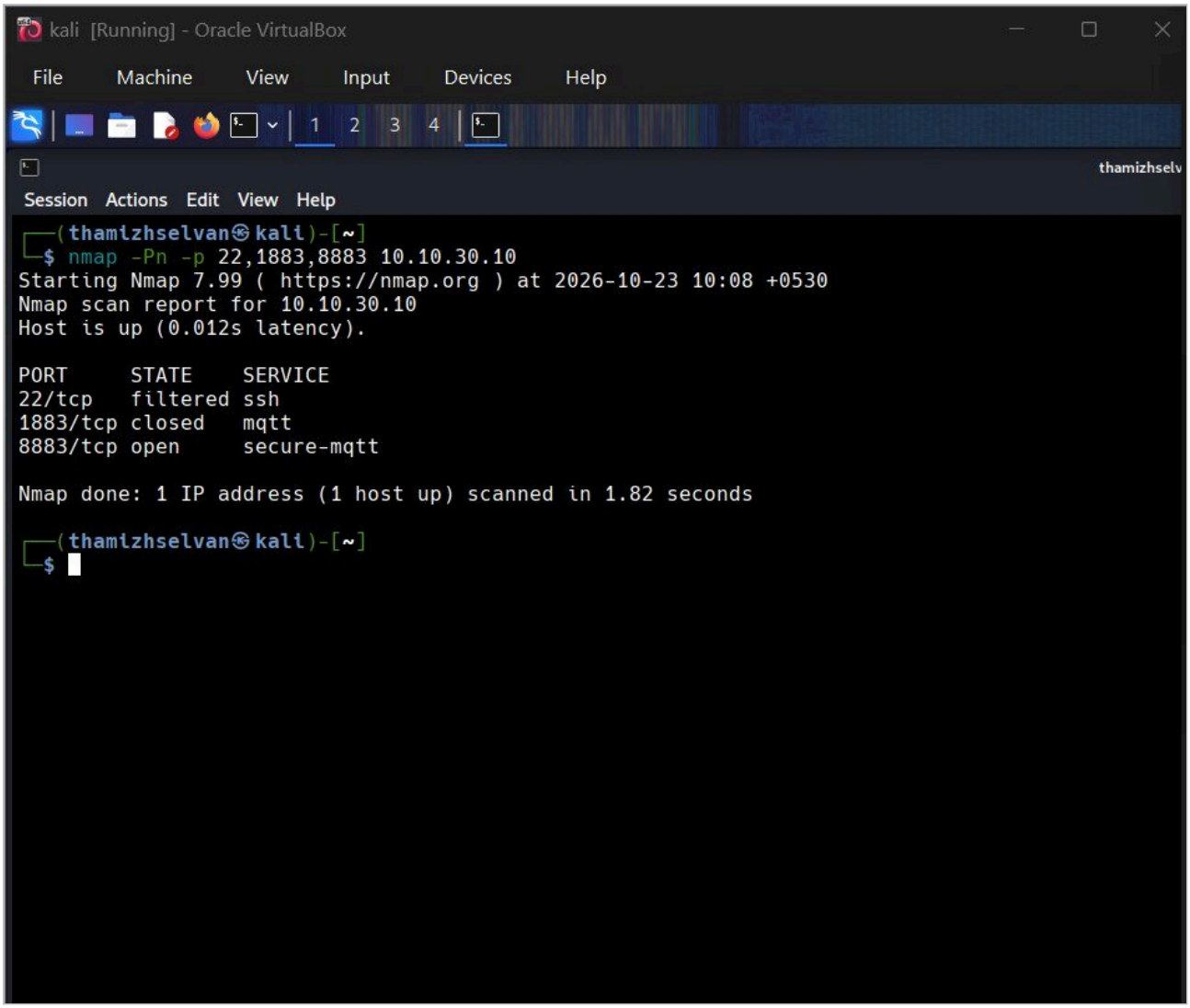


routes in place, then the broker scan



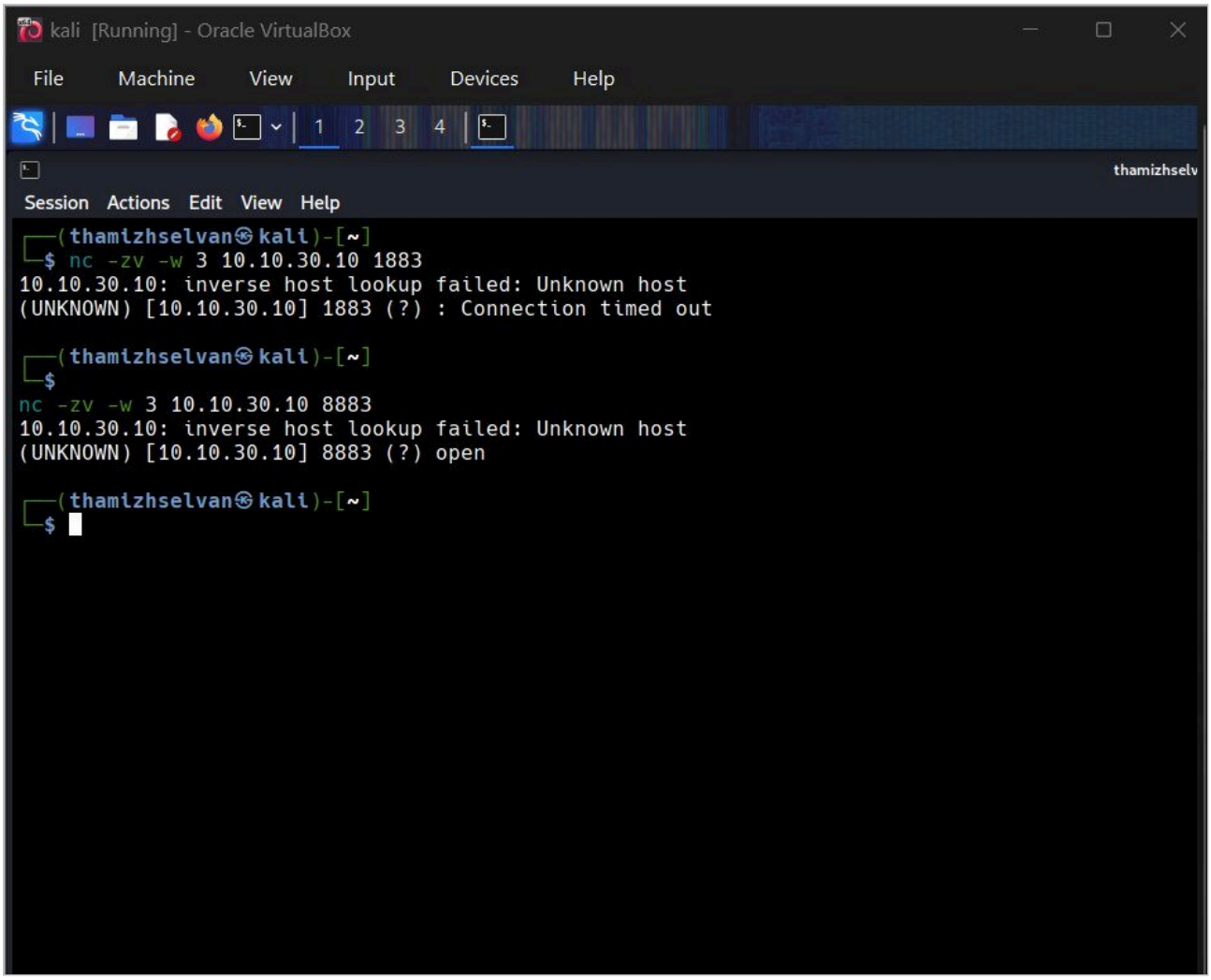
port 1883 refused, gateway SSH timed out, trusted-net scan filtered

The three ports that matter, on the broker. A targeted scan shows 8883 open (secure-mqtt), 1883 closed (nothing listens) and 22 filtered.



nmap of ports 22, 1883 and 8883 on the broker

After tightening the firewall. I changed the gateway forward rule from tcp 1883 and 8883 to 8883 only. Now the connection to 1883 times out, which means the firewall drops it, while 8883 still connects.

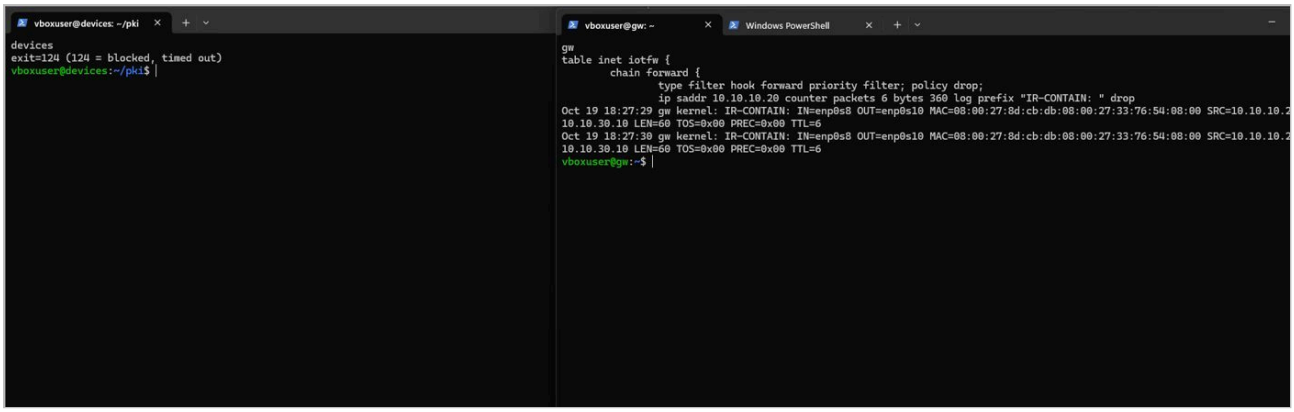


1883 times out, 8883 open

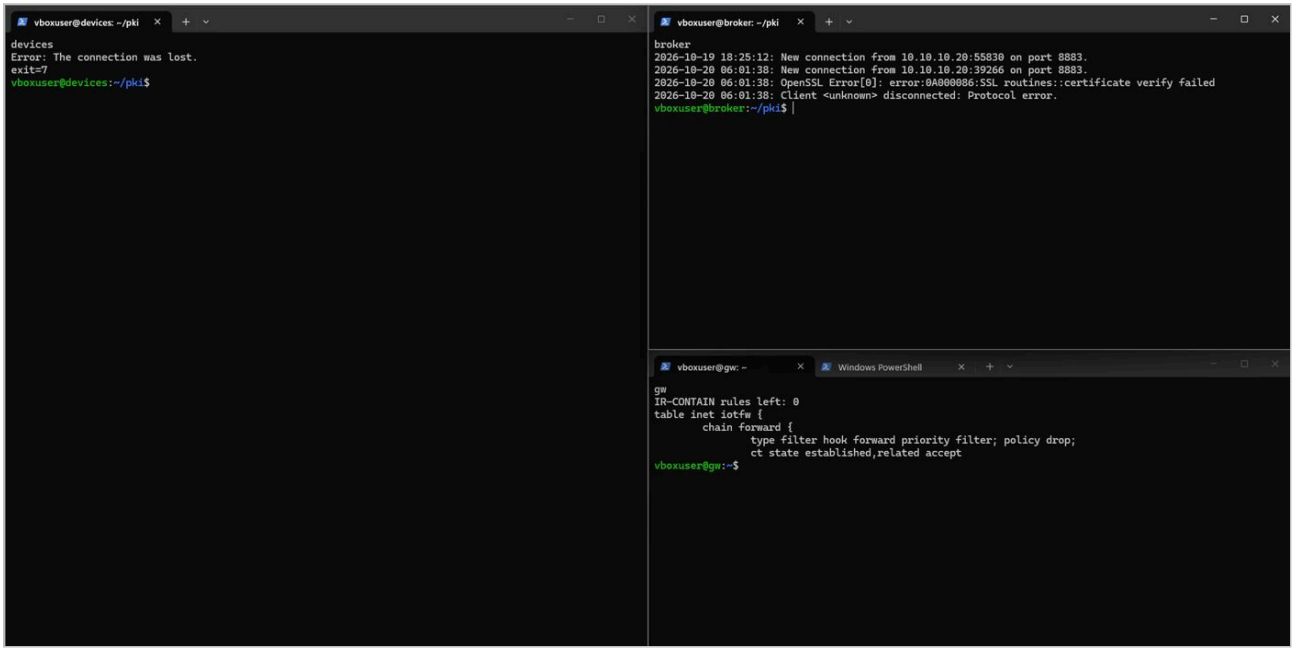
Incident response drill

I ran a drill where the thermo01 device is treated as compromised. The goal was to cut it off, make its credentials useless, and bring a clean identity back online.

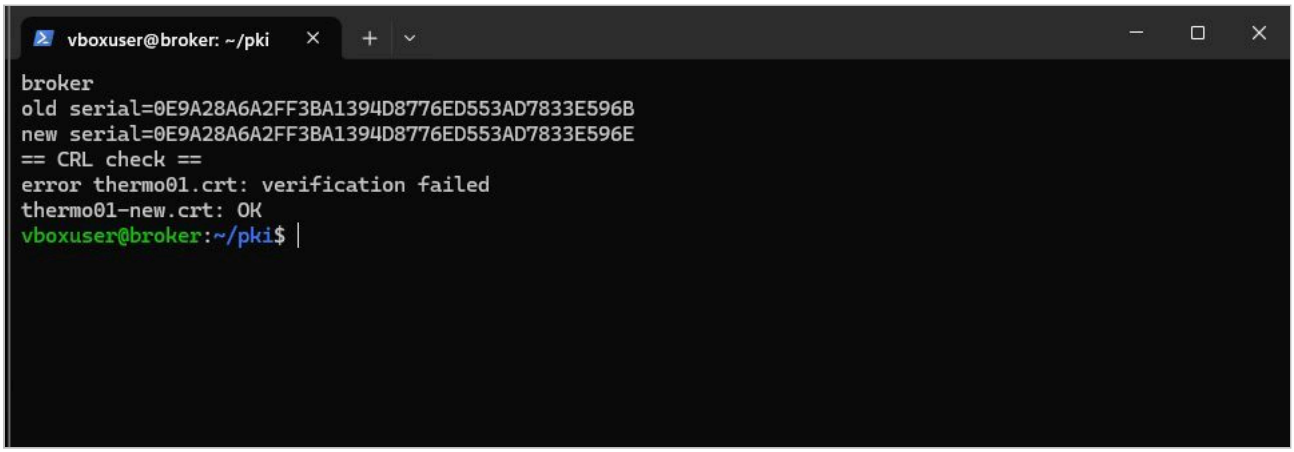
1. **Contain.** On the gateway I added an IR-CONTAIN drop rule for the device. A connection attempt then timed out (exit code 124).
2. **Revoke.** I added the device's old certificate to the CRL. A connection with the old certificate was refused.
3. **Reissue.** I issued a new certificate from IoT-Lab-CA. The old and new serial numbers differ, and the CRL check lists only the old one.
4. **Recover.** The device published again with the new certificate, while the old certificate stayed refused.
5. **Clean up.** I removed the containment rule (the count of IR-CONTAIN rules left is 0) and shredded the old private key.



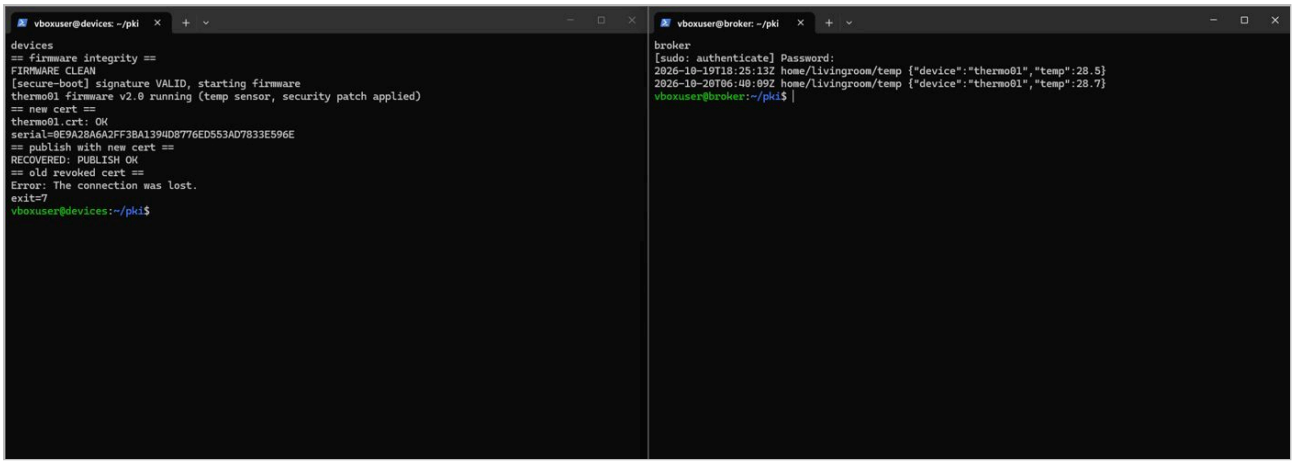
containment rule and timed-out connection



old certificate rejected, IR-CONTAIN rules left 0



old and new serials, CRL check



recovered publish with the new certificate

Incident response plan

The drill above followed this plan.

Phase	Action	Owner
Detect	Suricata alert, firewall drop log or tamper alert on home/security/alerts/	Administrator
Contain	Add a gateway drop rule for the device	Administrator
Eradicate	Revoke the device certificate and shred its key	Administrator
Recover	Issue a new certificate, confirm the device publishes again	Administrator
Review	Record what happened, remove the containment rule, tune the rules	Administrator and household

User training and awareness

The controls above only hold if the people using the devices do not undo them. The training covers what a household or small office needs to do, in plain language.

- Change default passwords on every device and the router, and use a unique one for each.
- Put smart devices on their own network, away from laptops and phones.
- Install firmware updates from the vendor only, and turn on automatic updates where offered.
- Turn off features you do not use, such as remote access and open ports.
- Know the signs of a problem: a device behaving oddly, unknown devices on the network, an unexpected lock or unlock.
- Know who to tell and what to do first: disconnect the device, then report it.

Awareness quiz

I wrote a five-question quiz from the project notes to check that the main points landed. It is a two-page handout with a name and date line and an answer key.

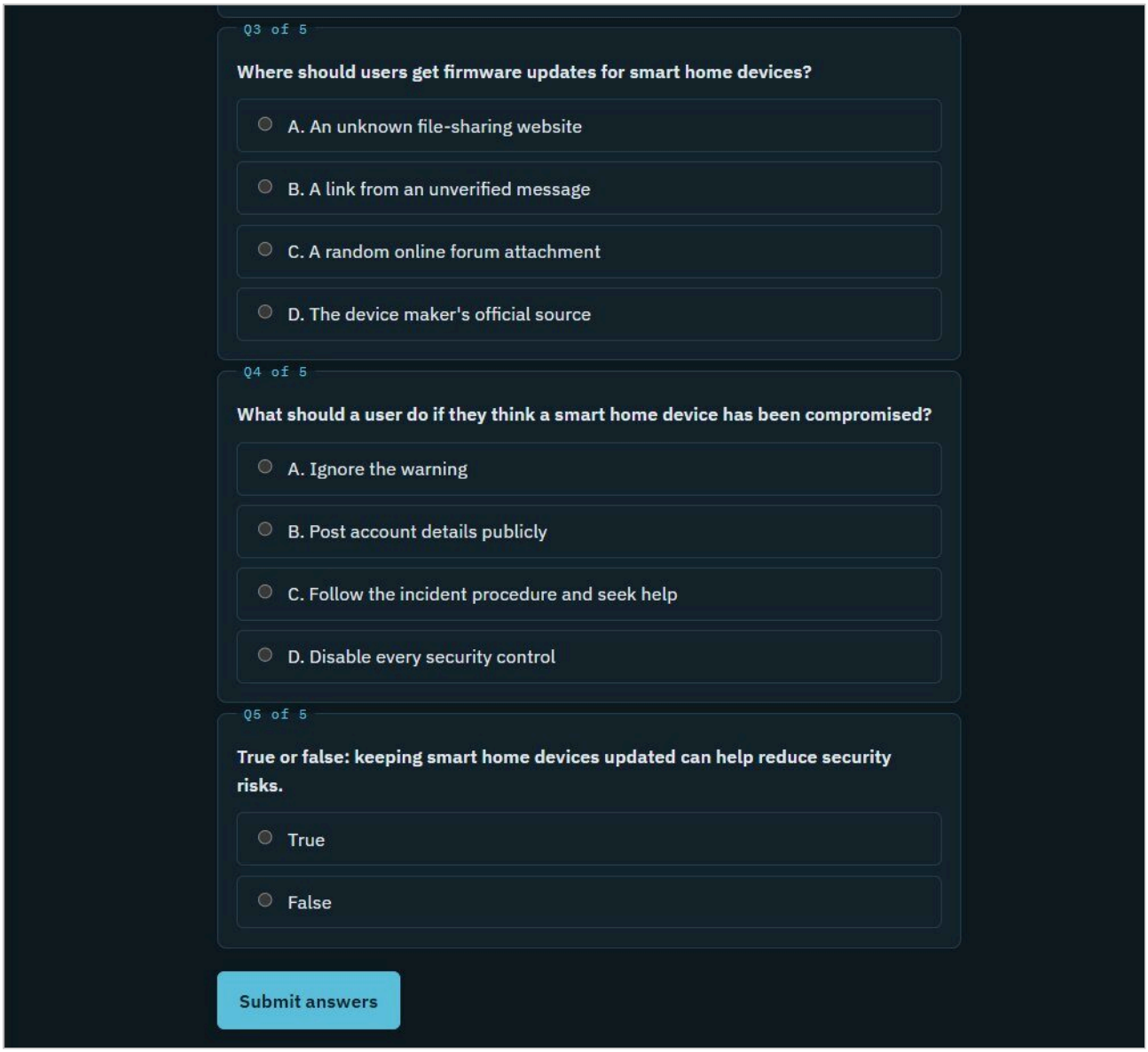
#	Question	Answer
1	Safest way to protect a smart home device account?	A strong, unique password
2	Which port is used for MQTT over TLS in this project?	8883
3	Where should firmware updates come from?	The device maker's official source
4	What to do if a device may be compromised?	Follow the incident procedure and seek help
5	True or false: keeping devices updated reduces risk.	True

Questions 2 and 4 tie back to the lab: 1883 is closed and 8883 is the only MQTT port, and the incident response drill is the procedure.

The quiz is also an online page, which is what my friends used. Each answer is scored straight away, with a short reason shown under it.



online quiz, questions 1 and 2



online quiz, questions 3 to 5

> **How it is delivered:** as written guidance plus an online quiz, not a live session. I sent the quiz link to friends, and each person answered on their own phone. The page scores the answers straight away and shows a short reason under each one, so a wrong answer also teaches the point.

Results. Four people took the quiz and sent back their scores: 5/5, 5/5, 4/5 and 3/5. The average was 4.25 out of 5 (17 of 20 points). The sample is small and these are friends, so I read it as a sanity check that the material is clear, not as a study.

Proposed schedule. Give the guidance and quiz when a device is installed. Repeat the quiz every six months, and again after any security incident or when a new type of device is added.

Policy and compliance

This lab is not a certified product, but each control maps to a recognised framework. I used the NIST Cybersecurity Framework functions and the GDPR security articles as the reference points.

Control in the lab	NIST CSF function	GDPR link
Network segmentation, default-drop firewall	Protect	Art. 32, security of processing
TLS with mutual certificates	Protect	Art. 32, encryption in transit
MQTT ACL	Protect	Art. 25, data protection by design, least privilege
LUKS2 vault, encrypted cache	Protect	Art. 32, encryption at rest
Signed firmware, secure update	Protect	Art. 32, integrity
Suricata, firewall logs, tamper alerts	Detect	Art. 33, needs detection to notice a breach
Containment, revocation, reissue	Respond, Recover	Art. 33, breach notification within 72 hours

Policy points I would adopt for a real deployment:

1. Every device gets its own certificate, and no shared credentials.
2. A compromised device is isolated first and its certificate revoked the same day.
3. Firmware is signed, and unsigned or older versions are never installed.
4. Logs are kept long enough to reconstruct an incident.
5. If personal data could be affected, the breach is assessed for notification under GDPR.

Policy area	Rule
Encryption	All device traffic uses TLS. Sensitive data on storage is encrypted with AES-256.
Authentication	Each device and administrator has its own certificate. Default and shared passwords are not allowed.
Authorization	Each identity gets the minimum topics it needs. Changes are made in the ACL and reviewed.
Updates	Firmware is signed and checked before install. Older versions are refused.
Incident response	Follow the plan above. Contain first, then revoke, reissue and review.

Conclusion, limitations and future work

Every attack that worked against the baseline failed against the hardened lab: anonymous connect, plaintext sniffing, a spoofed lock command, and open ports. The layers also back each other up. If the

firewall is bypassed, TLS and the ACL still stand; if a device is compromised, the IR drill shows how to contain it.

Limitations

- Secure boot and tamper detection are software simulations, not UEFI Secure Boot or hardware tamper switches.
- It is a VM lab with one simulated device. Real devices have constrained hardware, vendor firmware and cloud dependencies I did not test.
- Certificates are issued by my own CA. A real deployment needs a managed PKI.
- The NAT adapter on broker and devices exists for lab management only.

Future work

- Add more device types and a real hardware device.
- Automate certificate renewal and revocation.
- Send Suricata and firewall logs to a central SIEM for correlation.
- Test with a second attacker position, such as the trusted network.

Appendix: awareness quiz handout

A five-question quiz built from this project, handed to household members and friends. Name: _____ Date: _____

Q1. What is the safest way to protect a smart home IoT device account?

A. Keep the default password · B. Use a strong, unique password · C. Share the password with everyone · D. Use the same simple password everywhere

Q2. In our project, which port is used for MQTT over TLS?

A. 1883 · B. 21 · C. 80 · D. 8883

Q3. Where should users get firmware updates for smart home devices?

A. An unknown file-sharing website · B. A link from an unverified message · C. A random online forum attachment · D. The device maker's official source

Q4. What should a user do if they think a smart home device has been compromised?

A. Ignore the warning · B. Post account details publicly · C. Follow the incident procedure and seek help · D. Disable every security control

Q5. True or False: keeping smart home devices updated can help reduce security risks.

Answer key: Q1 B · Q2 D · Q3 D · Q4 C · Q5 True